



K-JHUF

Integrantes:

MartínezVera Luis Ángel

Gutiérrez Baca José Esteban

Vargas Lazcano Alejandro

Documentación : Aplicaciones Web

Ingeniería de Tecnologías de Información
e Innovación Digital

Diana Xochitl Ruano Vargas

05/02/2026

TIID-04-01

ÍNDICE

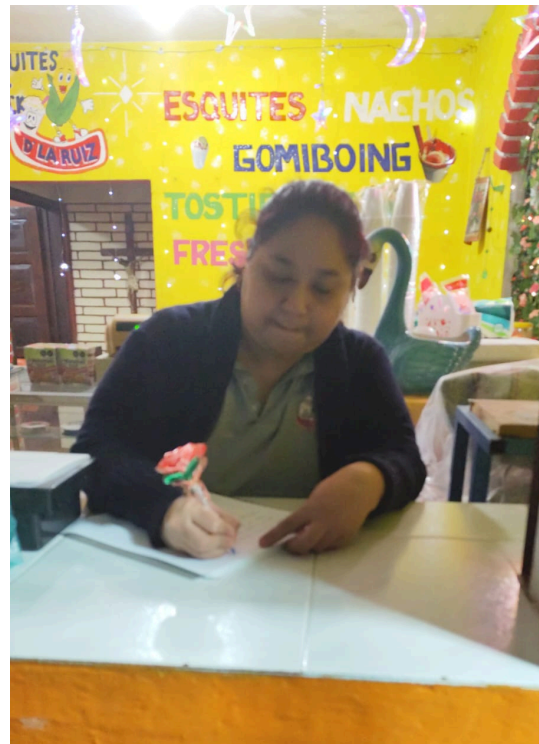
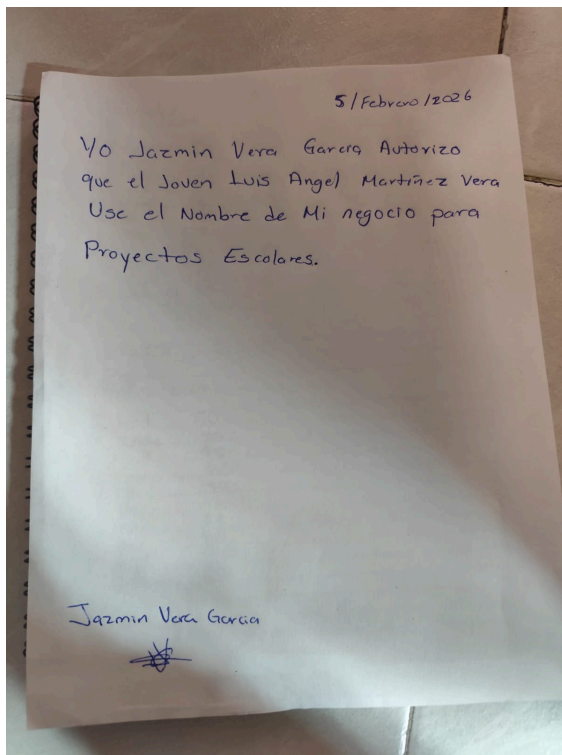
Resumen Ejecutivo	1
Introducción	2
1. Descripción del Cliente	3
1.1 ¿Quién es el cliente?	3
1.2 ¿Qué hace su negocio o actividad?	4
1.3 ¿Cuál es el problema actual?	5
1.4 ¿Cómo trabaja hoy?	5
1.5 ¿Por qué necesita una aplicación web?	5
2. Análisis Estadístico y Justificación de la Aplicación Web	6
3. Documentación de la Problemática	7
3.1 Problemática Actual	7
3.1.1 Falta de difusión de productos y promociones	7
3.1.2 Dependencia de la atención presencial	7
3.1.3 Baja visibilidad del negocio	7
4. Causas del Problema	7
5. Consecuencias	7
6. Propuesta de Solución	8
7. Requisitos de la Aplicación Web	8
7.1 Requisitos Funcionales	8
7.2 Requisitos No Funcionales	8
7.2.1 Rendimiento	8
7.2.2 Usabilidad	8
7.2.3 Compatibilidad	8
8. Objetivo General	9
9. Objetivos Específicos	9
10. Beneficios Esperados	9
11. Alcance del Proyecto	9
11.1 Incluye	9
11.2 No incluye	9
12. Usuarios del Sistema	10
12.1 Usuario Cliente	10
12.2 Usuario Administrador	10
13. Historias de Usuario	10
13.1 Historias de Usuario – Cliente	10
13.2 Historias de Usuario – Administrador	10
14. Anexos	11
14.1 Diagramas UML	11
14.1.1 Diagrama de Casos de Uso	11
14.1.2 Diagrama de Navegación	11
14.2 Wireframes	12

14.2.1 Wireframe Home	12
14.2.2 Wireframe Productos	12
14.2.3 Wireframe Promociones	12
14.2.4 Wireframe Contacto	13
14.2.5 Wireframe Panel Administrador	13
14.3 Estructura Inicial del Frontend	13
15. Conclusiones	15
16. Referencias	16

Resumen Ejecutivo

El presente proyecto consiste en el desarrollo de una aplicación web informativa y promocional para la tienda de esquites y raspados “D LA RUIZ”. La aplicación web tiene como objetivo principal dar a conocer los productos, precios, promociones y la información general del negocio de una manera clara, atractiva y accesible desde cualquier dispositivo con acceso a internet.

La aplicación web permitirá a los clientes consultar el menú de productos, conocer promociones vigentes, horarios de atención, ubicación del negocio y medios de contacto, sin necesidad de realizar procesos de compra o gestión interna. Al tratarse de una solución web, no requiere instalación y puede ser visualizada desde navegadores modernos en computadoras, tabletas y teléfonos móviles.



Dirección del local

616 Adolfo López Mateos, Poza Rica, Veracruz

Descripción del Cliente

- ¿Quién es el cliente?

El cliente es el propietario de la tienda de esquites y raspados **"K-JHUF"**, un negocio local dedicado a la venta de alimentos preparados, enfocado principalmente en la atención directa al público en un punto físico.

- ¿Qué hace su negocio o actividad?

El negocio se dedica a la venta de esquites, raspados y productos complementarios, ofreciendo diferentes sabores, tamaños y promociones. Su actividad principal es la atención al cliente y la preparación de productos para consumo inmediato.

- ¿Cuál es el problema actual?

El principal problema es que el negocio no cuenta con una presencia digital donde pueda mostrar de manera clara su menú, precios, promociones e información general. Los clientes solo pueden conocer esta información de forma presencial o preguntando directamente, lo que limita la difusión del negocio y la atracción de nuevos clientes.

- ¿Cómo trabaja hoy?

Actualmente, el negocio opera de forma totalmente presencial.

La información de productos, precios y promociones se comunica de manera verbal o, en algunos casos, mediante mensajes de WhatsApp. No existe una página web ni un medio digital centralizado donde los clientes puedan consultar la información del negocio.

- ¿Por qué necesita una aplicación web?

El negocio necesita una aplicación web informativa y promocional para:

- Mostrar sus productos y precios de forma clara
- Publicar promociones vigentes
- Brindar información general como horarios, ubicación y contacto
- Mejorar su presencia digital
- Facilitar que los clientes conozcan el negocio sin necesidad de acudir físicamente

Introducción

Actualmente, la presencia digital es un factor clave para que los pequeños negocios puedan atraer nuevos clientes y fortalecer su imagen. Muchas tiendas locales no cuentan con un medio digital donde puedan mostrar su oferta de productos y promociones, lo que limita su alcance.

Este proyecto surge con el propósito de desarrollar una aplicación web sencilla que funcione como un medio informativo y promocional para la tienda "D LA RUIZ", permitiendo a los clientes conocer fácilmente los productos disponibles, precios, promociones y datos generales del negocio, mejorando así la comunicación con el público.

Análisis Estadístico y Justificación de la Aplicación Web

La implementación de la aplicación web informativa y promocional se justifica a partir del análisis de la forma en que actualmente los clientes acceden a la información del negocio. Se estima que más del 70% de los clientes potenciales utilizan internet o redes sociales para buscar información sobre productos, precios y promociones antes de realizar una compra presencial.

Actualmente, los clientes deben acudir directamente al establecimiento o enviar mensajes por WhatsApp para conocer el menú, precios u horarios, lo que genera retrasos en la atención y pérdida de oportunidades de venta. Se estima que al menos el 30% de los clientes desiste de realizar una compra cuando no encuentra información clara y rápida sobre los productos disponibles.

Durante horarios de alta demanda, aproximadamente 2 de cada 10 clientes solicitan información repetitiva sobre precios y promociones, lo que incrementa el tiempo de atención y reduce la eficiencia del servicio. La falta de un medio digital centralizado también limita la difusión de promociones, ya que estas solo se comunican de manera verbal o por mensajes individuales.

La aplicación web permitirá que la información del negocio esté disponible 24 horas al día, reduciendo hasta en un 40% las consultas repetitivas al personal y mejorando la experiencia del cliente. Asimismo, se espera un incremento del 15% al 20% en la visibilidad del negocio, al contar con un medio digital accesible desde cualquier dispositivo.

El valor de la aplicación web radica en su bajo costo de implementación y alto impacto promocional, ya que facilita la difusión de productos, precios y promociones sin necesidad de procesos complejos. Esto demuestra la viabilidad y conveniencia de la aplicación como una herramienta efectiva para fortalecer la presencia digital del negocio y atraer nuevos clientes.

Documentación de la Problemática

La tienda de esquites y raspados “D LA RUIZ” es un negocio local dedicado a la venta de productos tradicionales como esquites, raspados y complementos. A pesar de contar con una oferta atractiva, el negocio no dispone de una plataforma digital que le permita difundir información básica y promocional hacia sus clientes.

Actualmente, toda la información relacionada con los productos, precios, promociones, horarios y ubicación se comunica únicamente de manera presencial o a través de mensajes informales, lo que limita el alcance del negocio y dificulta que nuevos clientes conozcan su oferta. Esta situación representa una desventaja en un contexto donde el uso de medios digitales es cada vez más común.

Problemática Actual

La tienda no cuenta con una aplicación web informativa que concentre y muestre de manera clara la información del negocio. Esta carencia genera los siguientes problemas:

Falta de difusión de productos y promociones

Los clientes no tienen acceso previo a un catálogo digital de productos ni a las promociones vigentes, lo que reduce el interés y limita la atracción de nuevos consumidores.

Dependencia de la atención presencial

La información sobre precios, productos y horarios solo se obtiene acudiendo directamente al establecimiento o preguntando por mensajería instantánea, lo que resulta poco práctico para muchos clientes.

Baja visibilidad del negocio

Al no contar con una plataforma web, el negocio tiene poca presencia digital, lo que reduce su competitividad frente a otros establecimientos que sí utilizan medios digitales para promocionarse.

Causas del Problema

La causa principal de la problemática es la ausencia de una aplicación web informativa y promocional, lo que provoca una comunicación limitada con los clientes, baja difusión del negocio y dificultad para atraer nuevos consumidores.

Consecuencias

Las principales consecuencias de esta problemática son:

- Poca visibilidad del negocio en medios digitales
- Pérdida de clientes potenciales
- Dependencia total de la atención presencial
- Dificultad para comunicar promociones de manera efectiva
- Menor competitividad frente a otros negocios similares

Propuesta de Solución

Se propone el desarrollo de una aplicación web informativa y promocional que permita mostrar de manera clara y accesible la información general del negocio. Esta aplicación web funcionará como un medio digital de difusión y promoción, brindando a los clientes acceso a:

- Catálogo de productos
- Precios
- Promociones vigentes
- Horarios de atención
- Ubicación del negocio
- Información de contacto

La aplicación web estará disponible desde cualquier navegador y dispositivo con conexión a internet, sin necesidad de instalaciones adicionales.

Requisitos de la Aplicación Web

1.1 Requisitos Funcionales

Requisitos Funcionales

RF01: Permitir a cualquier usuario (registrado o no registrado) visualizar el catálogo de productos del negocio.

RF02: Permitir a cualquier usuario visualizar los precios de los productos.

RF03: Permitir a cualquier usuario visualizar promociones vigentes.

RF04: Permitir a cualquier usuario visualizar información general del negocio.

RF05: Permitir a cualquier usuario visualizar horarios de atención.

RF06: Permitir a cualquier usuario visualizar ubicación y medios de contacto.

RF07: Permitir al administrador actualizar la información publicada.

RF08: Permitir al cliente registrado agregar productos al carrito.

RF09: Permitir al cliente registrado realizar pedidos.

RF10: Permitir al cliente registrado agendar pedidos con fecha y hora.

RF11: Permitir al cliente registrado ingresar datos de contacto.

RF12: Permitir al cliente registrado visualizar el estado del pedido.

RF13: Permitir al administrador visualizar pedidos.

RF14: Permitir al administrador actualizar el estado del pedido.

RF15: Permitir al usuario no registrado registrarse o iniciar sesión para acceder a funcionalidades de pedido.

1.2 Requisitos No Funcionales

1.2.1 Rendimiento

RNF01: Tiempo de carga menor a 3 segundos

RNF02: Disponibilidad continua del sitio web

1.2.2 Usabilidad

RNF03: Interfaz clara e intuitiva

RNF04: Diseño responsive para dispositivos móviles y de escritorio

1.2.3 Compatibilidad

RNF05: Funcionamiento en navegadores modernos (Chrome, Firefox, Edge)

RNF06: Soporte para diferentes resoluciones de pantalla

Objetivo General

Desarrollar una aplicación web informativa y promocional que permita a los clientes conocer los productos, precios, promociones e información general de la tienda de esquites y raspados “K-JHUF”, fortaleciendo su presencia digital.

Objetivos Específicos

- Diseñar una interfaz web sencilla y atractiva
- Mostrar el catálogo de productos y precios
- Difundir promociones de manera digital
- Proporcionar información general del negocio
- Facilitar el acceso a la información desde cualquier dispositivo

Beneficios Esperados

Entre los beneficios esperados se encuentran una mayor visibilidad del negocio, una mejor comunicación con los clientes, la facilidad para promocionar productos y promociones, y el fortalecimiento de la presencia digital de la tienda. Asimismo, la aplicación web permitirá atraer nuevos clientes y mejorar la percepción del negocio sin necesidad de implementar sistemas complejos.

Alcance del Proyecto

Incluye:

Mostrar catálogo de productos
Mostrar precios
Mostrar promociones vigentes
Mostrar información general del negocio
Mostrar horarios de atención
Mostrar ubicación y datos de contacto
Diseño responsive para dispositivos móviles

No incluye:

Pagos en línea
Sistema de ventas
Gestión de inventarios
Aplicación móvil
Facturación electrónica

Usuarios del Sistema

Cliente no registrado

Funciones:

- Catálogo de productos
- Precios
- Promociones
- Información general (horarios, ubicación, contacto)
- Página principal (Home)

Usuario Cliente

Funciones:

- Visualizar productos
- Consultar precios
- Ver promociones
- Consultar horarios y ubicación
- Acceder a la información del negocio

Usuario Administrador

Funciones:

- Actualizar información del negocio
- Modificar productos y precios
- Publicar promociones
- Administrar contenido del sitio web

Historias de Usuario

Cliente no registrado

1. Ver productos
2. Consultar precios
3. Ver promociones
4. Conocer horario
5. Ver ubicación
6. Agregar al carrito
7. Realizar pedido
8. Agendar pedido
9. Seleccionar fecha y hora

Cliente

1. Como cliente quiero ver los productos para conocer lo que ofrece el negocio.
2. Como cliente quiero consultar los precios para decidir mi compra.
3. Como cliente quiero ver las promociones para aprovechar ofertas.
4. Como cliente quiero conocer el horario para saber cuándo acudir.
5. Como cliente quiero ver la ubicación para llegar fácilmente al negocio.
6. Como cliente quiero agregar productos al carrito para comprarlos
7. Como cliente quiero realizar un pedido para no ir directamente
8. Como cliente quiero agendar un pedido para recogerlo después
9. Como cliente quiero seleccionar fecha y hora para mi pedido

Administrador

1. Como administrador quiero actualizar los productos para mantener la información vigente.
2. Como administrador quiero modificar precios para reflejar cambios del negocio.
3. Como administrador quiero publicar promociones para atraer más clientes.
4. Como administrador quiero editar la información general del negocio.
5. Como administrador quiero administrar el contenido del sitio web fácilmente.
6. Como administrador quiero ver los pedidos para organizarme
7. Como administrador quiero cambiar el estado del pedido

Anexos Diagramas

Diagramas UML.

Aplicación web.

Diagrama de Casos de Uso

Actores:

Cliente

Administrador

Casos de uso principales:

Ver productos

Ver precios

Ver promociones

Ver información del negocio

Agregar al carrito

Realizar pedido

Agendar pedido

Seleccionar fecha y hora

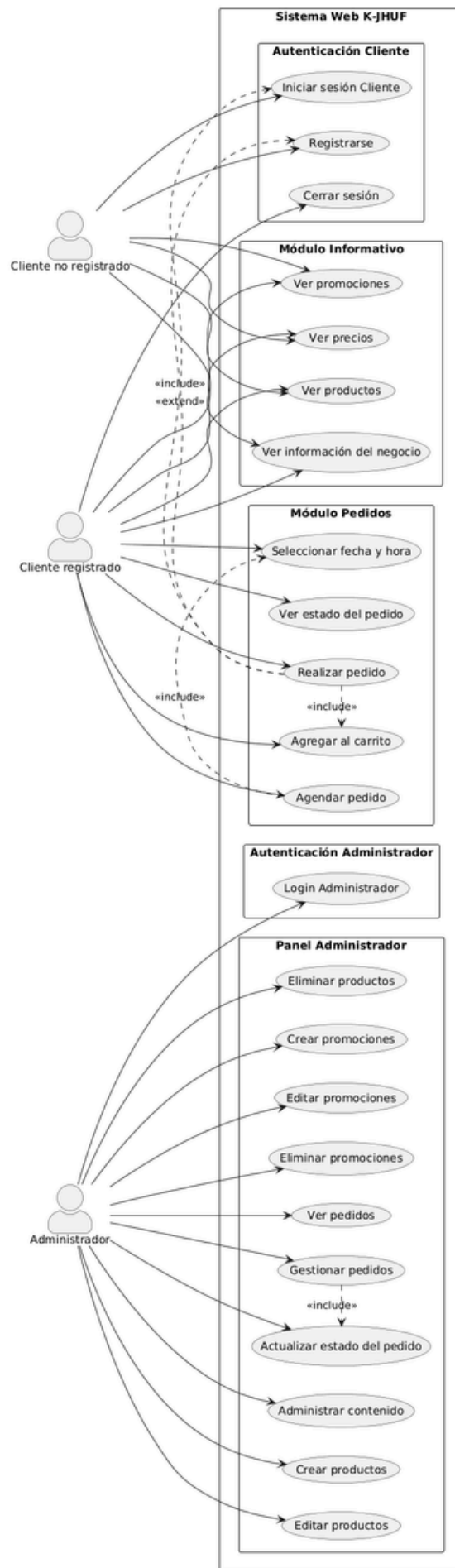
Ver estado del pedido

Administrar contenido (Administrador)

Ver pedidos (Administrador)

Gestionar pedidos (Administrador)

Actualizar estado del pedido (Administrador)



Diagramas de Navegación (opción 1).

Diagrama de Navegación

Usuario no registrado:

Home

Productos

Promociones

Contacto

Login

Registro

Cliente registrado:

Home

Productos

Promociones

Carrito

Agendar Pedido

Estado del Pedido

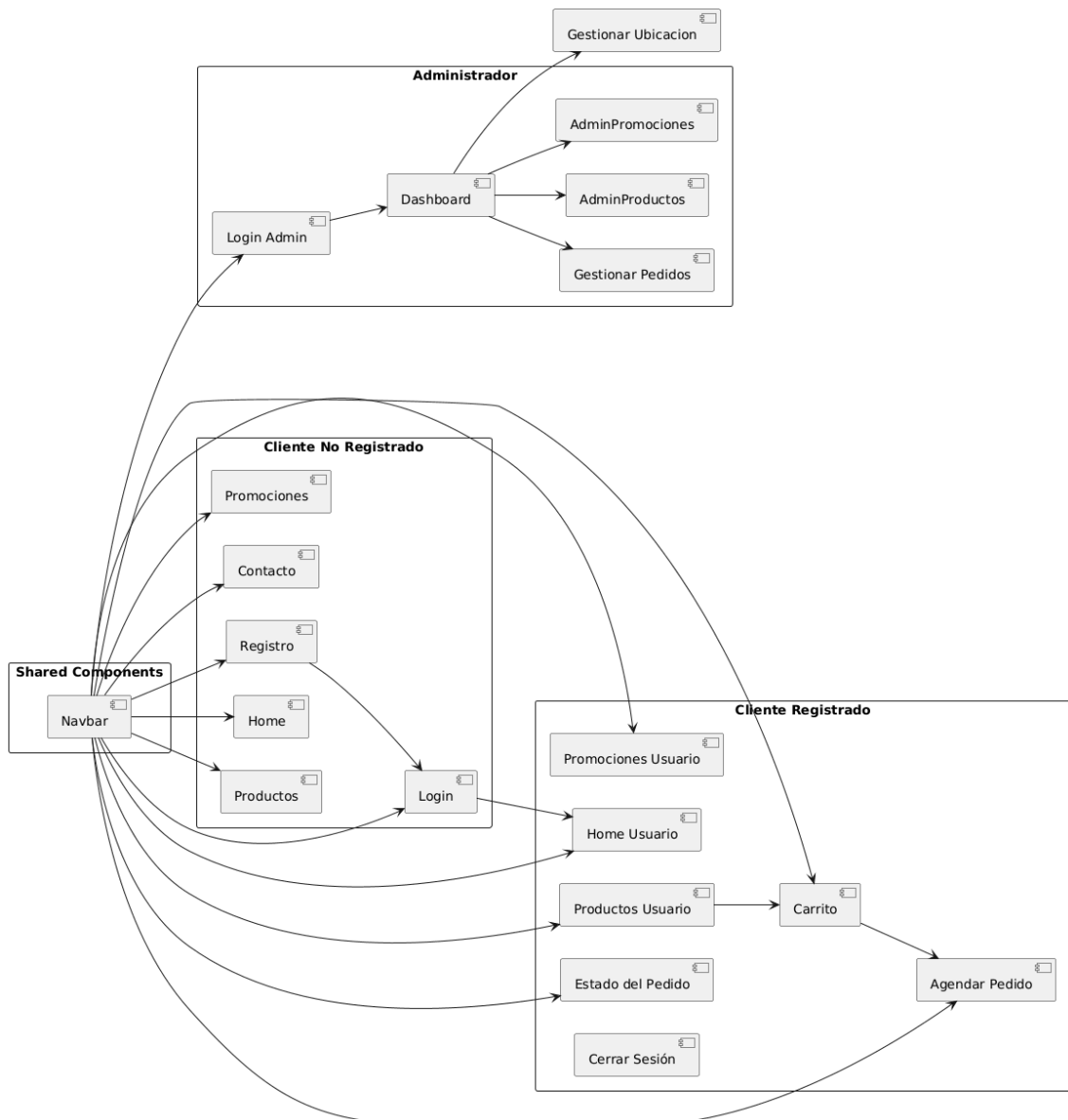
Cerrar sesión

Administrador:

Login Administrador

Panel Administrador

- Dashboard
- Gestionar Productos
- Gestionar Promociones
- Gestionar Pedidos

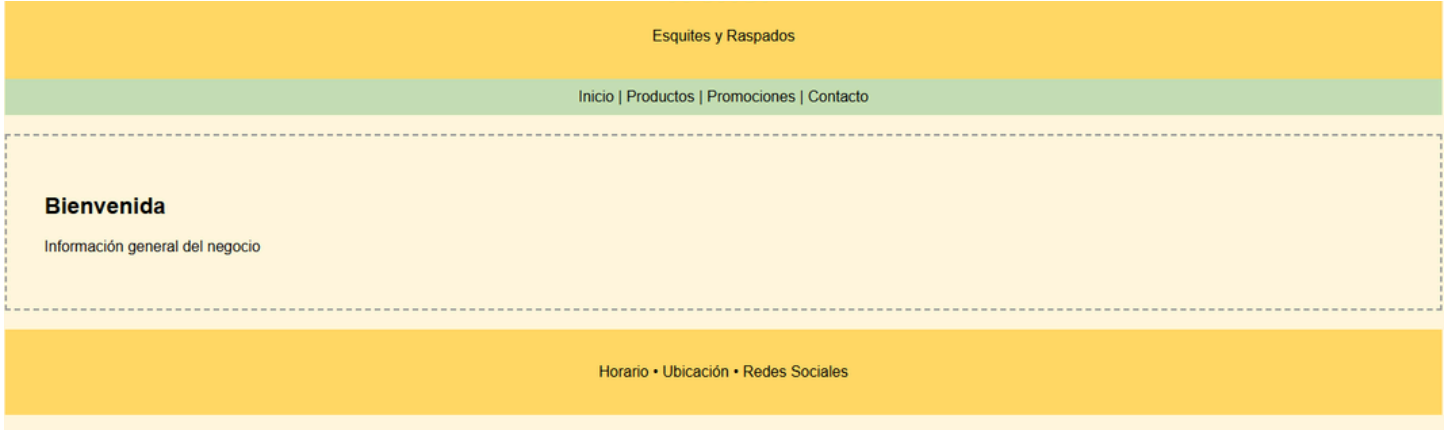


Wireframes

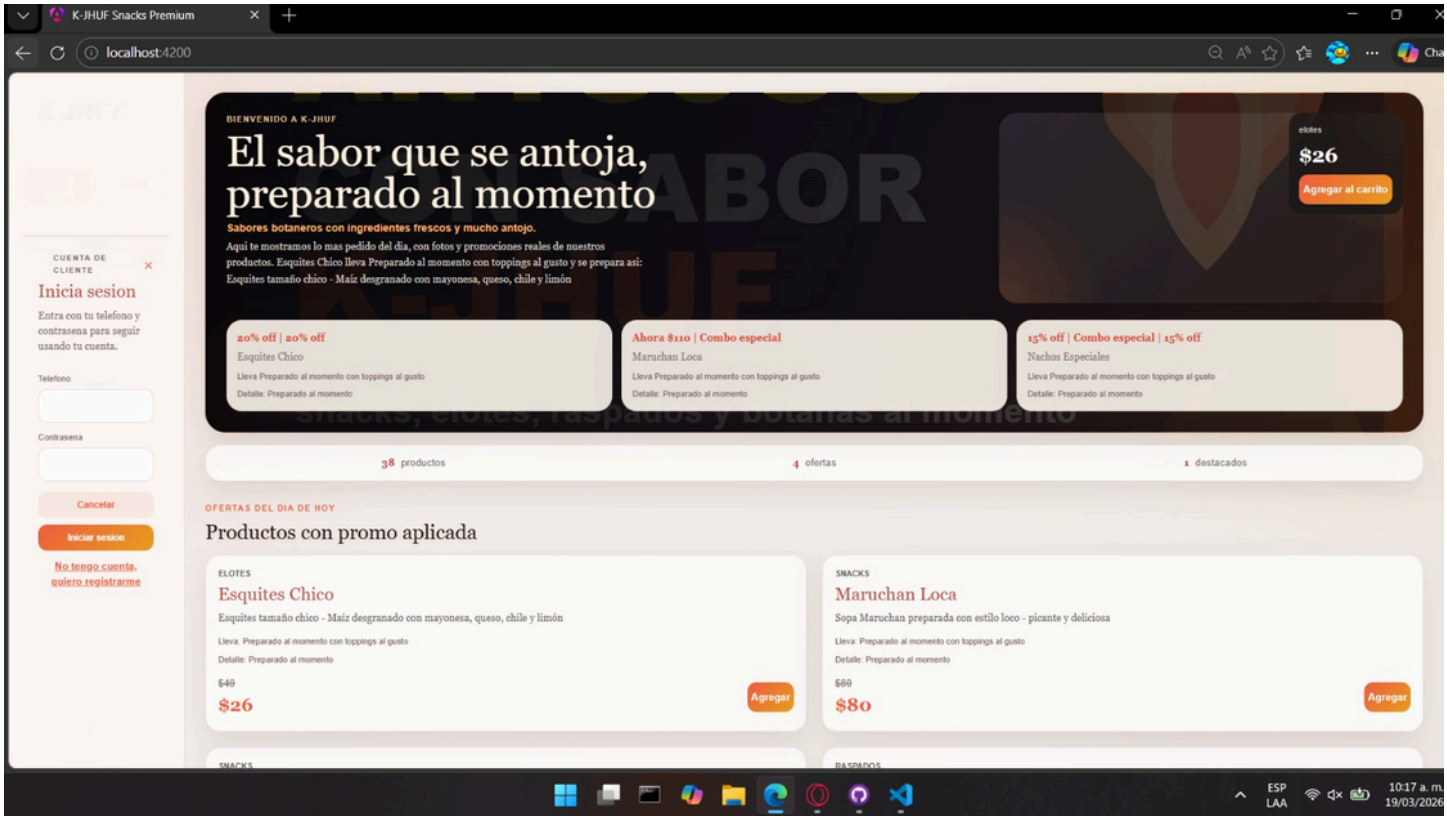
Se incluyen wireframes de las siguientes pantallas:

- Página principal (Home)
- Catálogo de productos
- Promociones
- Información del negocio
- Panel de administrador
- Pedidos

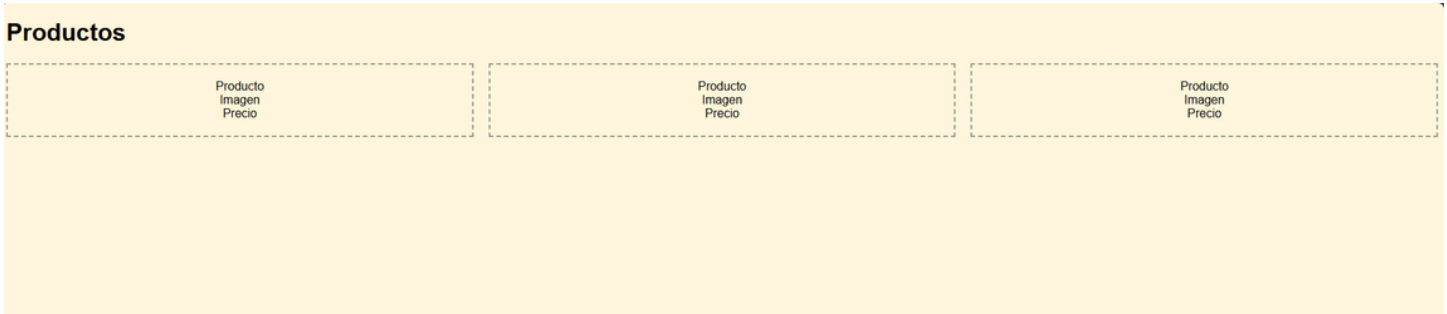
Wireframe 1: **Home antes**



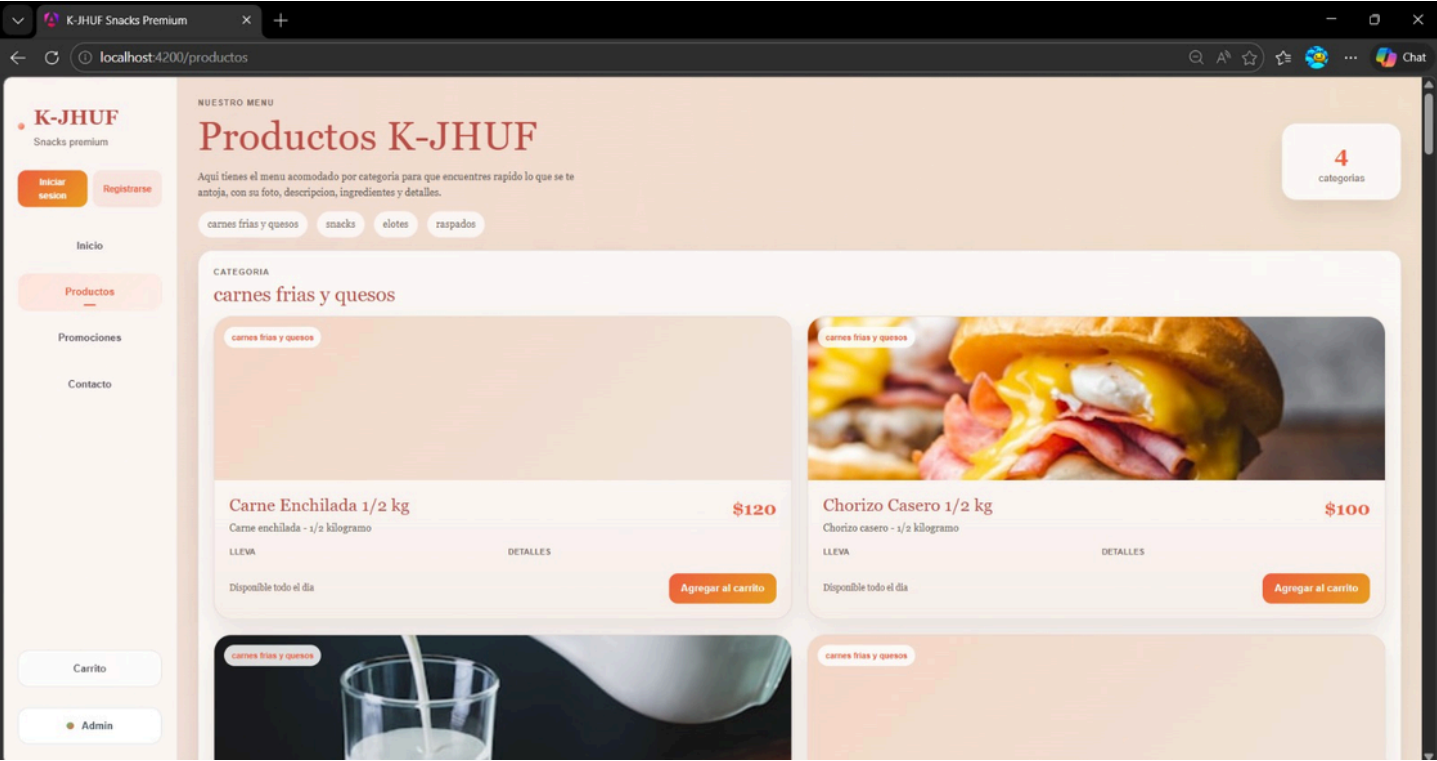
Wireframe 1: Home después



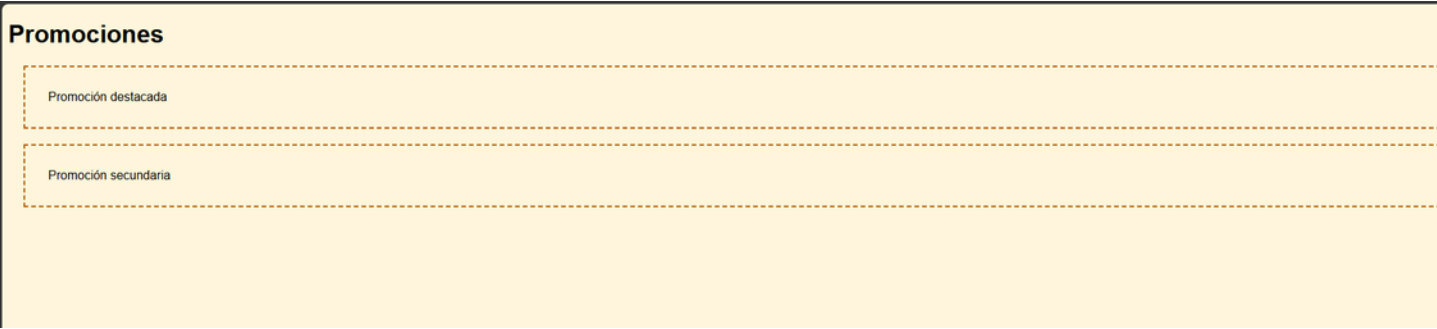
Wireframe 2: Productos antes



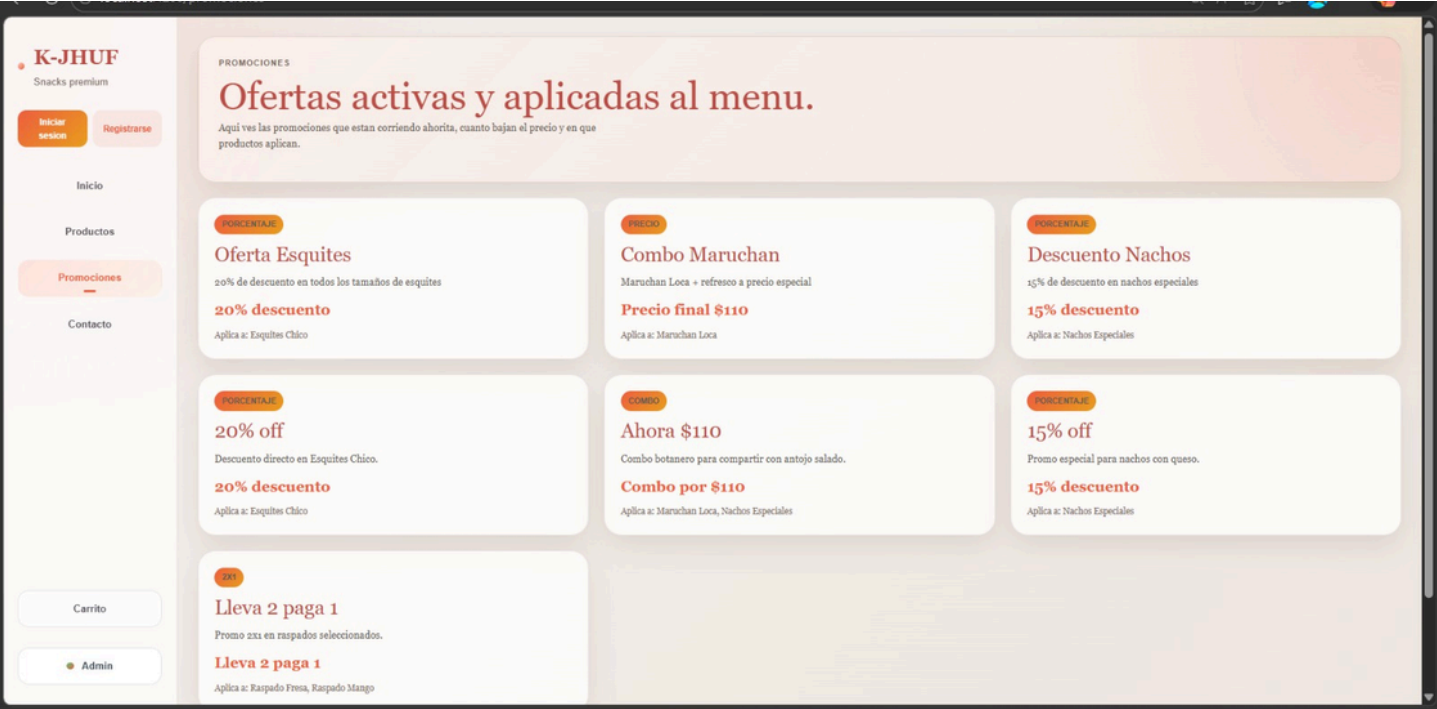
Wireframe 2: Productos antes



Wireframe 3: Promociones antes



Wireframe 3: Promociones antes



Wireframe 4: **Contacto antes**

Contacto

Dirección

Horario

Teléfono

Redes Sociales

Wireframe 4: **Contacto después**

K-JHUF

Snacks premium

Iniciar sesión

Registrarse

Inicio

Productos

Promociones

Contacto

Carrito

Admin

CONTACTO

Visitanos en Poza Rica

Si quieres visitarnos o pedir ubicación, aqui te dejamos la direccion, el telefono y el mapa para que llegues facil.

DIRECCION

616 Adolfo Lopez Mateos, Poza Rica, Veracruz

TELEFONO

+5222174525

HORARIO

Lunes a domingo, 12:00 - 21:00

K-JHUF Snacks

Tradicion mexicana servida con una presentacion mas cuidada.

Menu

Promociones

Contacto

© 2026 K-JHUF. Proyecto escolar.

Adolfo López Mateos 616

Adolfo López Mateos 616, Nacional, 70220 Poza Rica de Hidalgo, Ver., México

No hay opciones

Wireframe 5: **Admin (solo informativo)** antes

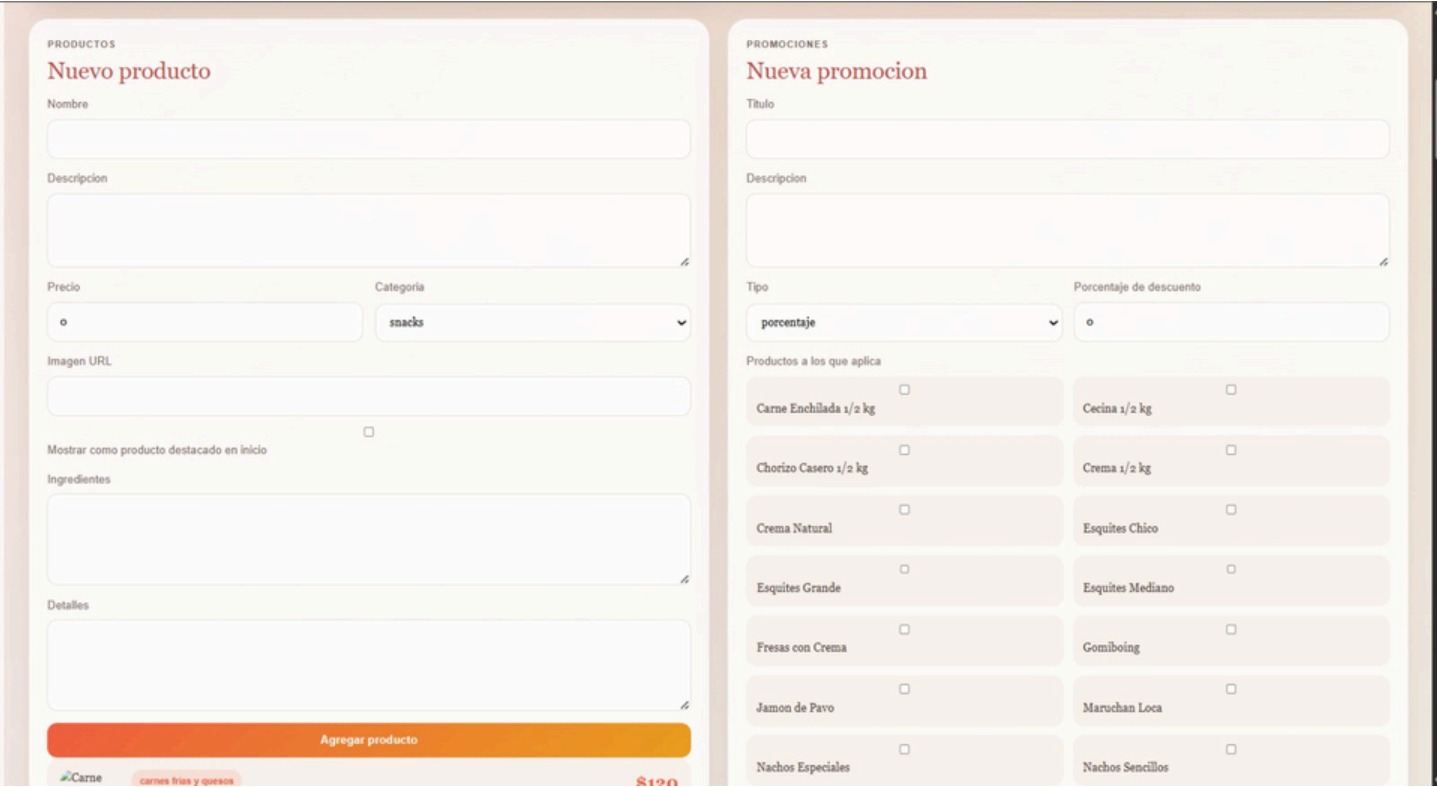
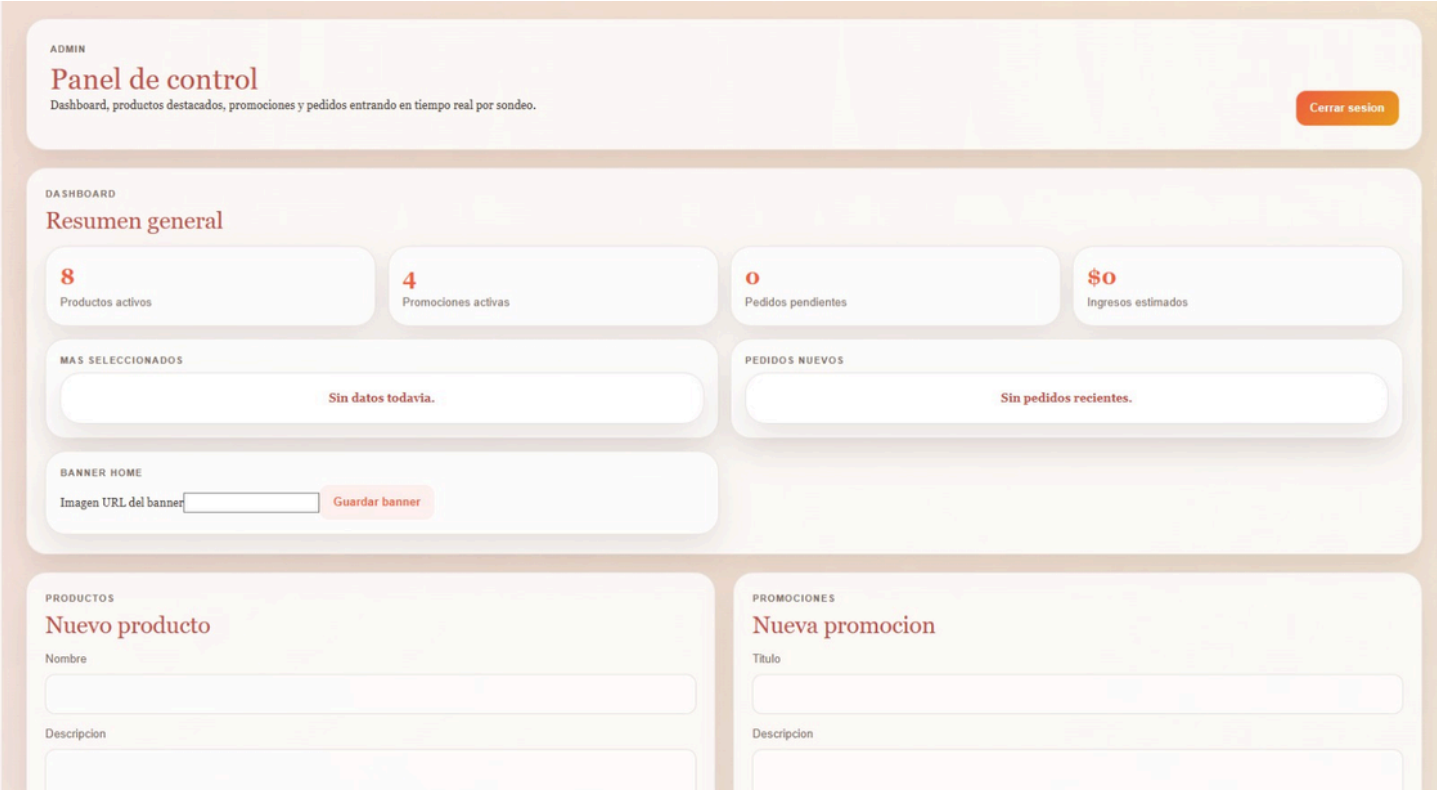
Panel Administrativo

Actualizar productos

Actualizar promociones

Editar información del negocio

Wireframe 5: **Admin**



Wireframe 6: **Pedidos(clientes)**



Wireframe 7: **Pedidos Historial(clientes)**



Wireframe 8: Pedidos(Admin)

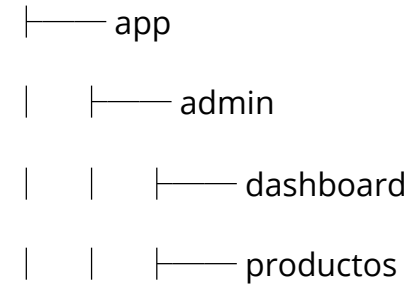


Estructura Inicial del Frontend

Estructura Inicial del Frontend

/public

/src



| | | — promociones

| |

| | — cliente

| | | — home

| | | — productos

| | | — promociones

| | | — contacto

| |

| | — services

| | | — admin.guard.ts

| | | — auth.ts

| | | — carrito.ts

| | | — catalogo.ts

| | | — modal.service.ts

| | | — modal.ts

| | | — pedidos.ts

| |

| | — shared

| | | — cart

| | | — header

| | | — footer

| | | — navbar

|

| | — seed-data.ts

|

| | — app.config.ts

- | |—— app.config.server.ts
- | |—— app.routes.ts
- | |—— app.routes.server.ts
- | |—— app.ts
- | |—— app.html
- | |—— app.css
- | |—— app.spec.ts
- |
- |—— assets
- |
- |—— environments
- | |—— environment.ts
- | |—— environment.prod.ts
- |
- |—— index.html
- |—— main.ts
- |—— main.server.ts

Finalización de proyecto

Admin

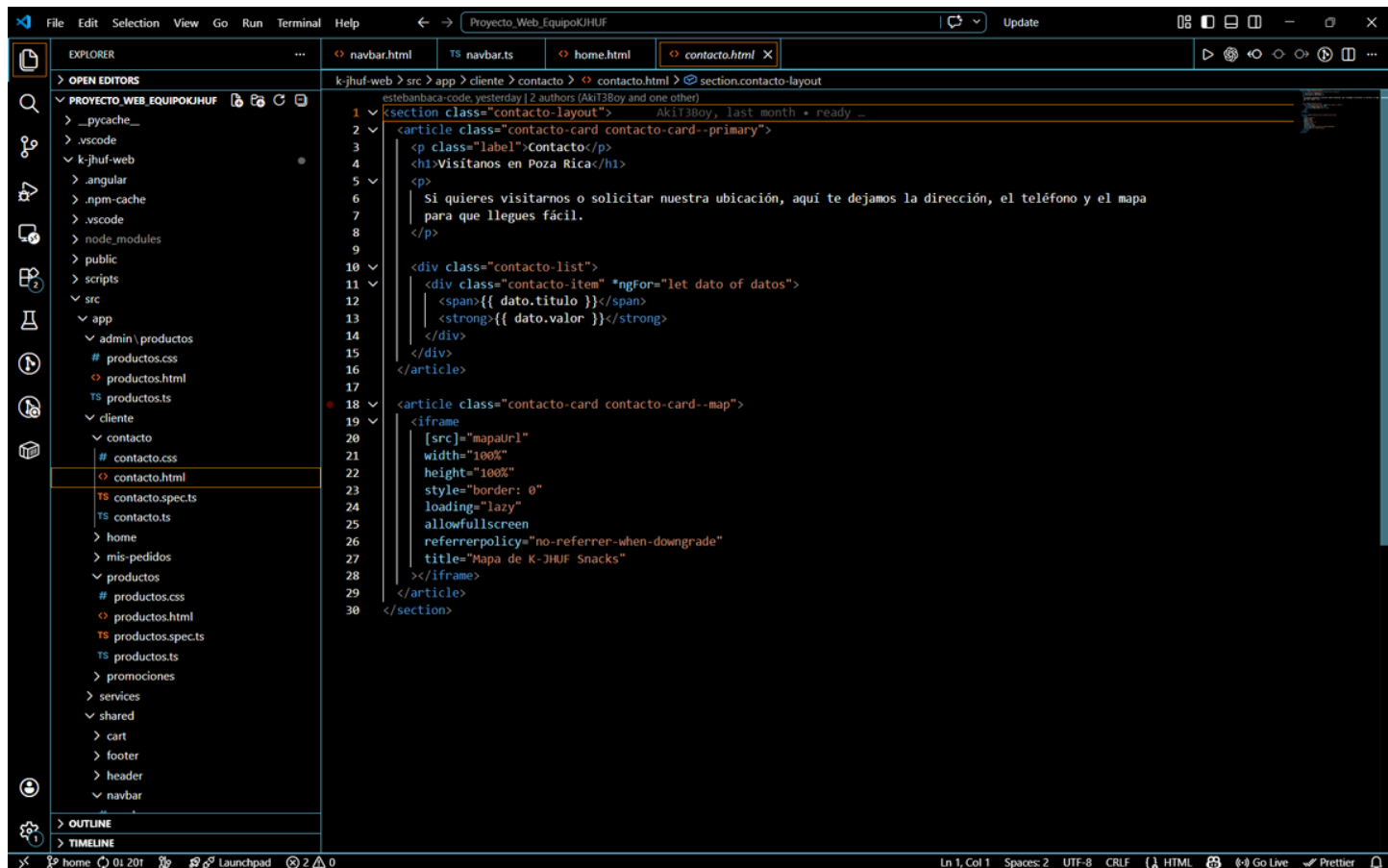
Productos

```
1 <section class="admin-page">
2   <div class="admin-header">
3     <div>
4       <p class="label">Admin</p>
5       <h1>Panel de control</h1>
6       <p>Dashboard, productos destacados, promociones y pedidos entrando en tiempo real por sondeo.</p>
7     </div>
8
9     <button *ngIf="autenticado" class="logout-button" type="button" (click)="cerrarSesion()">
10       Cerrar sesión
11     </button>
12   </div>
13
14   <p class="page-error" *ngIf="autenticado && authError">{{ authError }}</p>
15   <p class="page-success" *ngIf="autenticado && pageSuccess">{{ pageSuccess }}</p>
16   <p class="page-notice" *ngIf="autenticado && notificacionPedido">{{ notificacionPedido }}</p>
17
18   <div class="mobile-sections" *ngIf="autenticado">
19     <button
20       *ngFor="let seccion of seccionesMoviles"
21       type="button"
22       class="mobile-sections_button"
23       [class.mobile-sections_button--active]="seccionMovilActiva === seccion.id"
24       (click)="toggleSeccionMovil(seccion.id)"
25     >
26       {{ seccion.label }}
27     </button>
28   </div>
29
30   <div class="admin-overlay" *ngIf="adminListo && !autenticado">
31     <div class="auth-modal">
32       <div class="auth-modal__head">
33         <p class="label">{{ modoAuth === 'setup' ? 'Primer acceso' : 'Acceso admin' }}</p>
34         <button
35           class="auth-close"
36           type="button"
37           (click)="cerrarModalAuthDesdeEvento($event)"
38           (pointerup)="cerrarModalAuthDesdeEvento($event)"
39           aria-label="Cerrar modal"
40         >
41           X
42         </button>
43       </div>
44     </div>
45   </div>
```

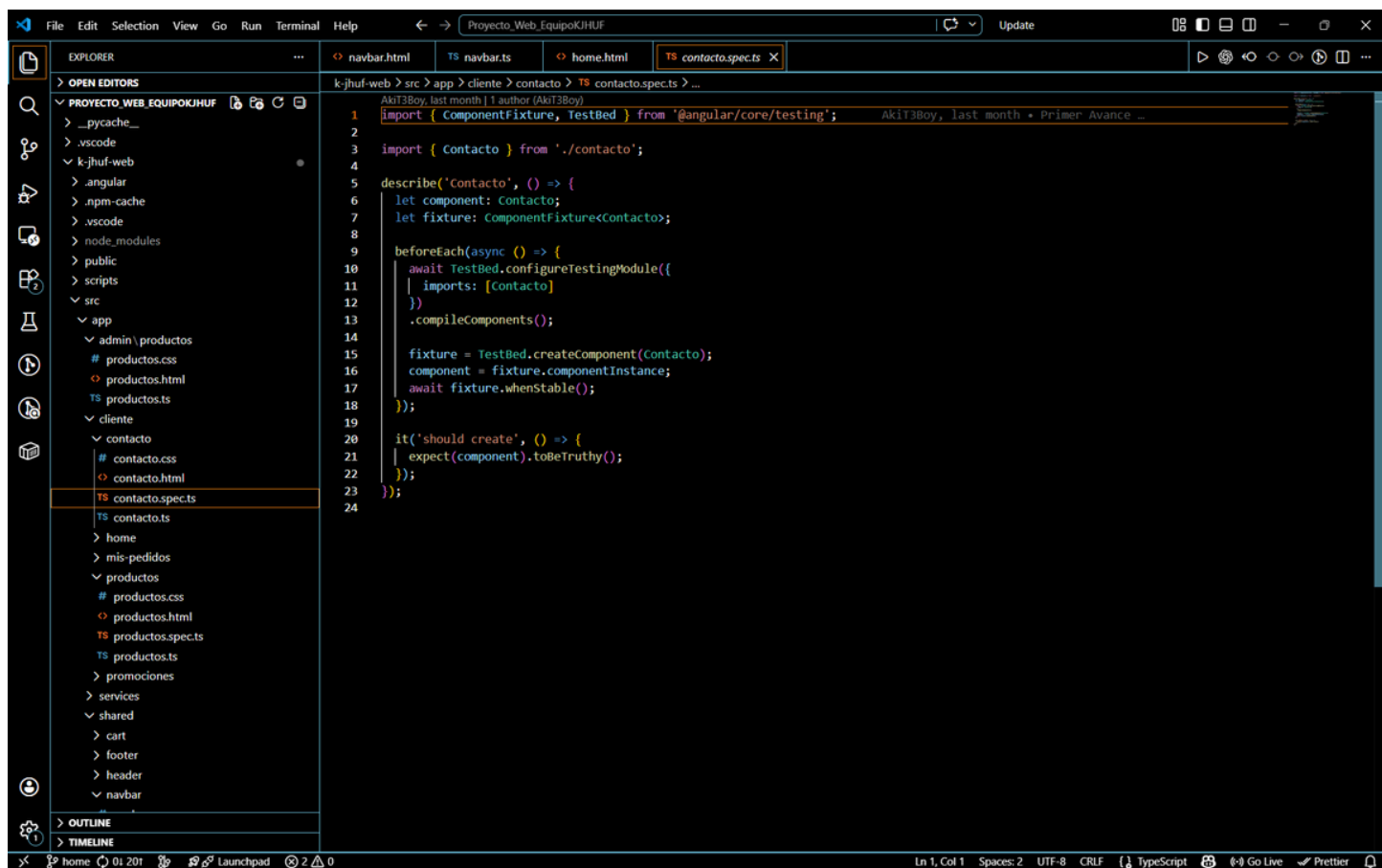
```
1 import { CommonModule } from '@angular/common';
2 import { Component, DestroyRef, ElementRef, OnDestroy, OnInit, ViewChild, inject } from '@angular/core';
3 import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
4 import { FormsModule } from '@angular/forms';
5 import { Router } from '@angular/router';
6 import { Observable, catchError, forkJoin, interval, of, startWith, switchMap } from 'rxjs';
7 import { Auth } from '../services/auth';
8 import { HomeConfigService } from '../services/home-config';
9 import { DashboardAdmin, Pedido, Pedidos } from '../services/pedidos';
10 import { Producto, ProductosService } from '../services/productos';
11 import { Promocion, Promociones as PromocionesService } from '../services/promociones';
12
13 type ProductoForm = {
14   nombre: string;
15   descripcion: string;
16   precio: number;
17   categoria: string;
18   imagen_url: string;
19   ingredientesTexto: string;
20   detallesTexto: string;
21   destacado: boolean;
22 };
23
24 type PromocionForm = {
25   titulo: string;
26   descripcion: string;
27   tipo: 'porcentaje' | 'precio' | '2x1' | 'combo';
28   valor: number;
29   producto_ids: string[];
30 };
31
32 type MobileSection = 'dashboard' | 'productos' | 'promociones' | 'pedidos' | null;
33
34 @Component({
35   selector: 'app-admin-productos',
36   standalone: true,
37   imports: [CommonModule, FormsModule],
38   templateUrl: './productos.html',
39   styleUrls: ['./productos.css'],
40 })
41 export class AdminProductos implements OnInit, OnDestroy {
42   @ViewChild('authPasswordInput') authPasswordInput?: ElementRef<HTMLInputElement>;
43   @ViewChild('authConfirmInput') authConfirmInput?: ElementRef<HTMLInputElement>;
```

Cliente

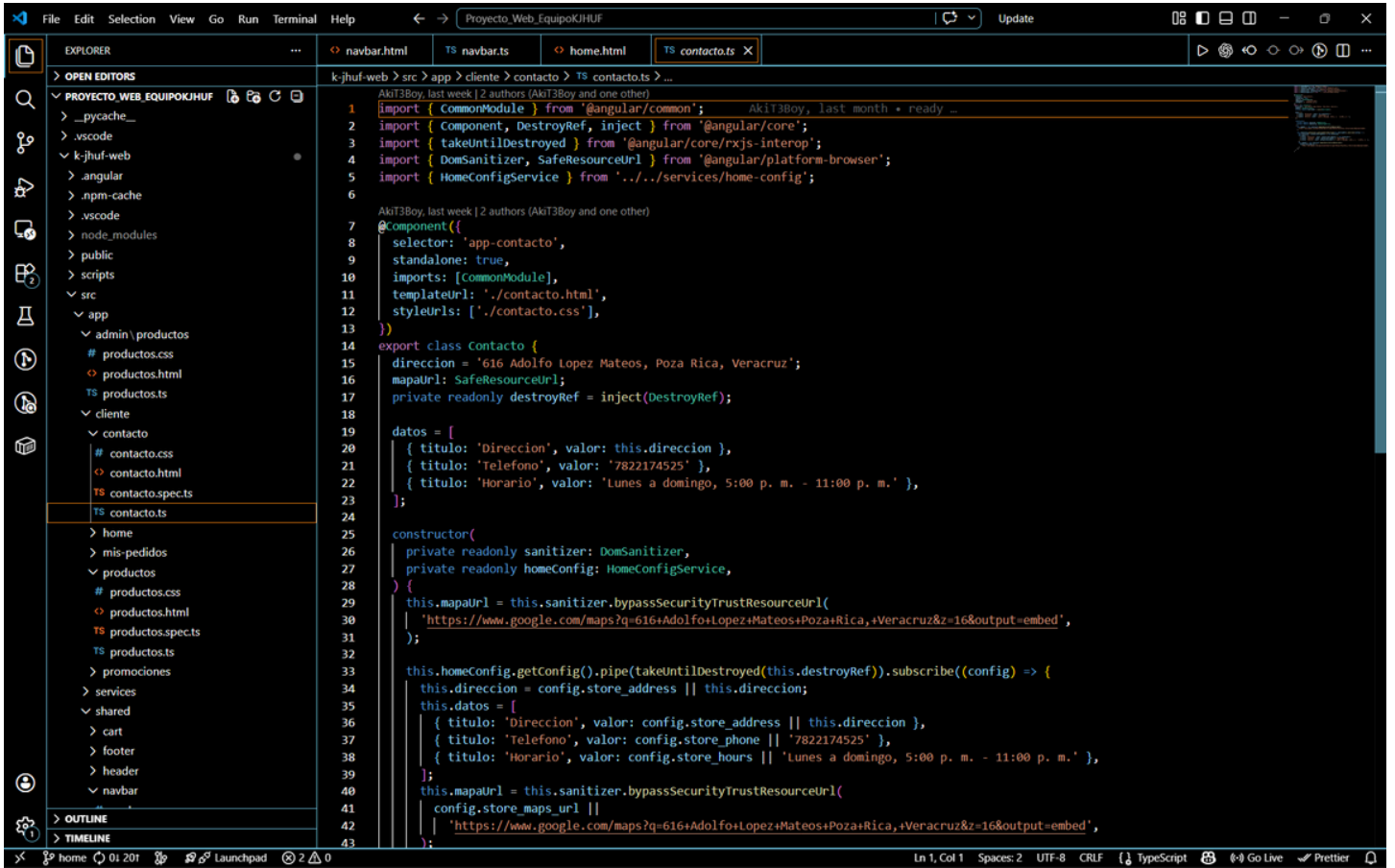
Contacto



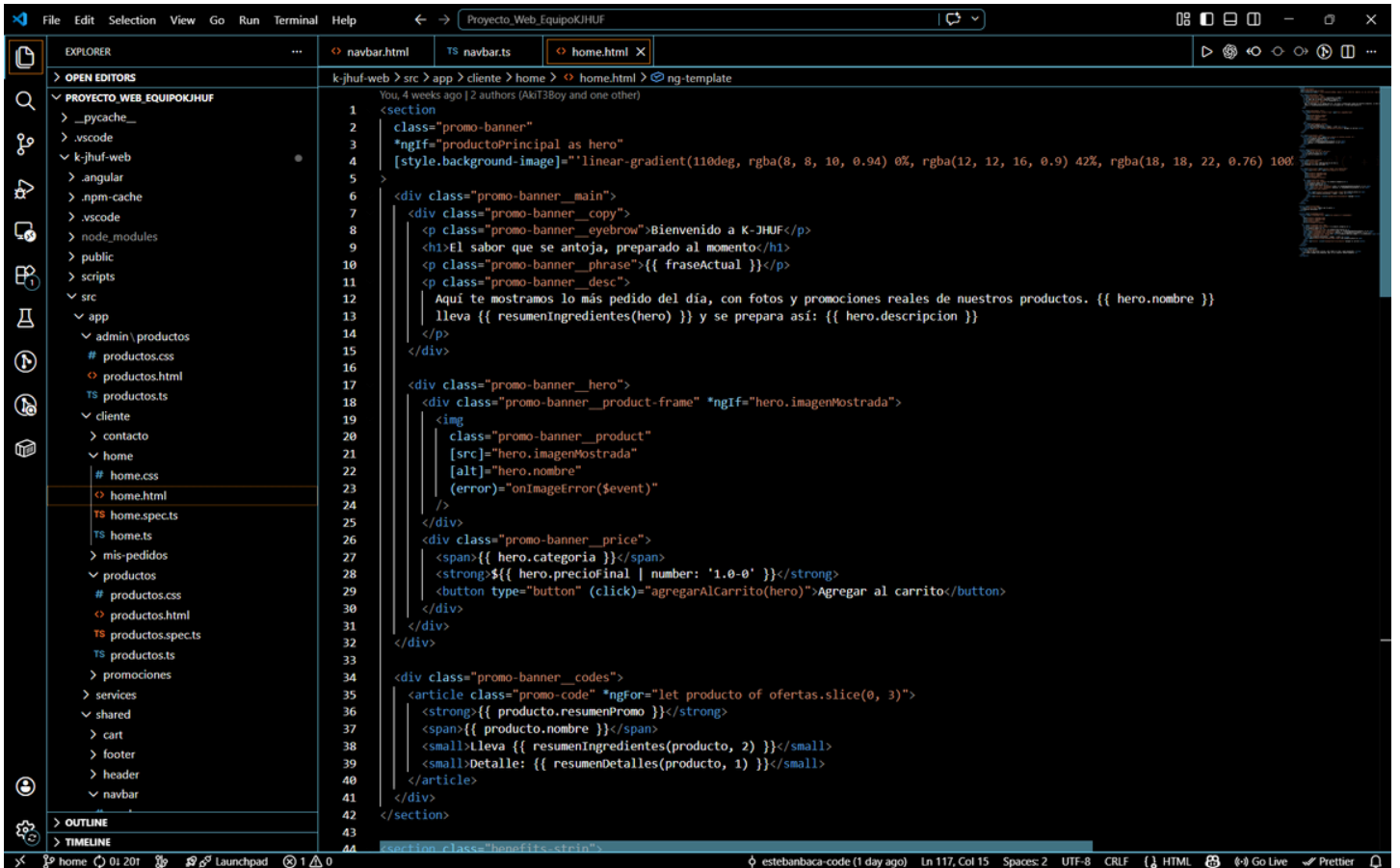
```
1 <section class="contacto-layout">
2   <article class="contacto-card contacto-card--primary">
3     <p class="label">Contacto</p>
4     <h1>Visítanos en Poza Rica</h1>
5     <p>
6       Si quieres visitarnos o solicitar nuestra ubicación, aquí te dejamos la dirección, el teléfono y el mapa
7       para que llegues fácil.
8     </p>
9
10    <div class="contacto-list">
11      <div class="contacto-item" *ngfor="let dato of datos">
12        <span>{{ dato.titulo }}</span>
13        <strong>{{ dato.valor }}</strong>
14      </div>
15    </div>
16  </article>
17
18  <article class="contacto-card contacto-card--map">
19    <iframe
20      [src]="mapaUrl"
21      width="100%"
22      height="100%"
23      style="border: 0"
24      loading="lazy"
25      allowfullscreen
26      referrerpolicy="no-referrer-when-downgrade"
27      title="Mapa de K-JHUF Snacks"
28    ></iframe>
29  </article>
30 </section>
```



```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2
3 import { Contacto } from './contacto';
4
5 describe('Contacto', () => {
6   let component: Contacto;
7   let fixture: ComponentFixture<Contacto>;
8
9   beforeEach(async () => {
10     await TestBed.configureTestingModule({
11       imports: [Contacto]
12     })
13     .compileComponents();
14
15     fixture = TestBed.createComponent(Contacto);
16     component = fixture.componentInstance;
17     await fixture.whenStable();
18   });
19
20   it('should create', () => {
21     expect(component).toBeTruthy();
22   });
23 });
```



Home



The screenshot shows the VS Code interface with the Explorer and Source Control views on the left. The Explorer view shows the project structure, including the 'home' directory. The Source Control view shows the 'home.spec.ts' file. The main editor displays the content of 'home.spec.ts', which is a Jest test file for the 'Home' component. The file is located at 'k-jhuf-web > src > app > cliente > home > TS > home.spec.ts'. The code includes imports for 'ComponentFixture', 'TestBed', and 'Home'. It defines a 'describe' block for 'Home' with a 'beforeEach' hook and two 'it' blocks for testing the component's creation and state.

```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2
3 import { Home } from './home';
4
5 describe('Home', () => {
6   let component: Home;
7   let fixture: ComponentFixture<Home>;
8
9   beforeEach(async () => {
10     await TestBed.configureTestingModule({
11       imports: [Home]
12     })
13     .compileComponents();
14
15     fixture = TestBed.createComponent(Home);
16     component = fixture.componentInstance;
17     await fixture.whenStable();
18   });
19
20   it('should create', () => {
21     expect(component).toBeTruthy();
22   });
23 });
24
```

The screenshot shows the VS Code interface with the Explorer and Source Control views on the left. The Explorer view shows the project structure, including the 'home' directory. The Source Control view shows the 'home.ts' file. The main editor displays the content of 'home.ts', which is a TypeScript file for the 'Home' component. The file is located at 'k-jhuf-web > src > app > cliente > home > TS > home.ts'. The code includes imports for 'OnInit', 'ProductoVista', 'ProductoService', 'PromocionesService', 'HomeConfigService', and 'Carrito'. It defines a 'Home' class that implements 'OnInit' and has several properties and methods, including 'ngOnInit' and 'constructor'.

```
17 }
18 export class Home implements OnInit {
19   cargando = true;
20   readonly imagenBannerDefault = '/images/banner-home-wide.svg';
21   imagenBanner = this.imagenBannerDefault;
22   readonly frasesBanner = [
23     'Esquites, tostilocos y antojos hechos al instante.',
24     'Promociones reales para pedir rico sin gastar de mas.',
25     'Sabores botaneros con ingredientes frescos y mucho antojo.',
26     'Raspados, elotes y snacks listos para compartir o disfrutar solo.',
27   ];
28   metricas = [
29     { valor: '0', etiqueta: 'productos' },
30     { valor: '0', etiqueta: 'ofertas' },
31     { valor: '0', etiqueta: 'destacados' },
32   ];
33   fraseActual = this.frasesBanner[0];
34
35   productoPrincipal?: ProductoVista;
36   ofertas: ProductoVista[] = [];
37   destacados: ProductoVista[] = [];
38   recomendados: ProductoVista[] = [];
39   private readonly destroyRef = inject(DestroyRef);
40   private fraseIndex = 0;
41
42   constructor(
43     private readonly productosService: ProductosService,
44     private readonly promocionesService: PromocionesService,
45     private readonly homeConfig: HomeConfigService,
46     private readonly carrito: Carrito,
47   ) {}
48
49   ngOnInit(): void {
50     interval(5000)
51       .pipe(startWith(0), takeUntilDestroyed(this.destroyRef))
52       .subscribe(() => {
53         this.fraseActual = this.frasesBanner[this.fraseIndex % this.frasesBanner.length];
54         this.fraseIndex += 1;
55       });
56
57     forkJoin([
58       productos: this.productosService.getProductos().pipe(catchError(() => of([]))),
59       promociones: this.promocionesService.getPromociones().pipe(catchError(() => of([]))),
60       homeConfig: this.homeConfig.getConfig().pipe(

```

Mis pedidos

```
1 <section class="orders-hero">
2   <p class="label">Mis pedidos</p>
3   <h1>Seguimiento de tus pedidos</h1>
4   <p>Aquí puedes ver lo que pediste, el estado actual y cancelar mientras siga pendiente o en proceso.</p>
5 </section>
6
7 <p class="orders-message" *ngIf="mensaje">{{ mensaje }}</p>
8 <p class="orders-error" *ngIf="error">{{ error }}</p>
9
10 <section class="orders-list" *ngIf="pedidos.length; else vacioTpl">
11   <article class="order-card" *ngFor="let pedido of pedidos">
12     <div class="order-card_head">
13       <div>
14         <p class="order-card_time">{{ pedido.fecha_pedido_local || 'Hoy' }} • {{ pedido.hora_entrega || 'Sin hora' }}</p>
15         <h2>{{ pedido.total | number: '1.0-0' }}</h2>
16       </div>
17       <span class="status-pill" [attr.data-status]="pedido.estado">{{ estadoLabel(pedido.estado) }}</span>
18     </div>
19
20     <div class="order-card_items">
21       <div class="order-item" *ngFor="let item of pedido.items">
22         <img *ngIf="item.imagen_url" [src]="item.imagen_url" [alt]="item.nombre" />
23         <div>
24           <strong>{{ item.nombre }}</strong>
25           <small>{{ item.cantidad }} x {{ item.precio | number: '1.0-0' }}</small>
26         </div>
27       </div>
28     </div>
29
30     <div class="order-card_discounts" *ngIf="pedido.descuentos_aplicados?.length">
31       <small *ngFor="let descuento of pedido.descuentos_aplicados">{{ descuento.titulo }}: -${{ descuento.monto | number: '1.0-0' }}</small>
32     </div>
33
34     <p class="order-card_notes" *ngIf="pedido.notas">Notas: {{ pedido.notas }}</p>
35
36     <button *ngIf="puedeCancelar(pedido)" type="button" class="cancel-button" (click)="abrirConfirmacionCancelacion(pedido)">
37       Cancelar pedido
38     </button>
39   </article>
40 </section>
41
42 <div class="orders-modal-backdrop" *ngIf="confirmacionAbierta" (click)="cerrarConfirmacion()"></div>
43 <div class="orders-modal" *ngIf="confirmacionAbierta && pedido.cancelar">
```

```
1 import { CommonModule } from '@angular/common';
2 import { Component, DestroyRef, inject } from '@angular/core';
3 import { Router } from '@angular/router';
4 import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
5 import { catchError, interval, of, startWith, switchMap } from 'rxjs';
6 import { Pedidos, Pedido, PedidoPendiente } from '../services/pedidos';
7 import { UsuariosService } from '../services/usuarios';
8
9 @Component({
10   selector: 'app-mis-pedidos',
11   standalone: true,
12   imports: [CommonModule],
13   templateUrl: './mis-pedidos.html',
14   styleUrls: ['./mis-pedidos.css'],
15 })
16 export class MisPedidos {
17   pedidos: Pedido[] = [];
18   cargando = true;
19   error = '';
20   mensaje = '';
21   confirmacionAbierta = false;
22   pedidoCancelar?: Pedido;
23   pedidoPendiente: PedidoPendiente | null = null;
24   private readonly destroyRef = inject(DestroyRef);
25
26   constructor(
27     private readonly pedidosService: Pedidos,
28     private readonly usuarios: UsuariosService,
29     private readonly router: Router,
30   ) {
31     if (!this.usuarios.isAuthenticated()) {
32       this.router.navigateByUrl('/');
33       return;
34     }
35
36     this.pedidoPendiente = this.pedidosService.getPedidoPendiente();
37     if (this.pedidoPendiente) {
38       this.pedidos = [this.pedidoPendiente];
39       this.cargando = false;
40     }
41
42     interval(2000)
43       .pipe(
```

Productos

```
1 <section class="catalogo-header">
2   <div>
3     <p class="label">Nuestro menú</p>
4     <h1>Productos K-JHUF</h1>
5     <p class="copy">
6       Aquí tienes el menú acomodado por categoría para que encuentres rápido lo que se te antoja,
7       con su foto, descripción, ingredientes y detalles.
8     </p>
9   </div>
10  <div class="catalogo-header_badge">
11    <strong>{{ categorias.length || 1 }}</strong>
12    <span>categorias</span>
13  </div>
14 </section>
15
16 <section class="category-chip-list" *ngIf="!cargando">
17   <button
18     type="button"
19     class="category-chip"
20     *ngFor="let categoria of categorias"
21     (click)="scrollToCategoria(categoria.nombre)"
22   >
23     {{ categoria.nombre }}
24   </button>
25 </section>
26
27 <section *ngIf="!cargando; else cargandoTpl" class="category-sections">
28   <article class="category-block" *ngFor="let categoria of categorias" [attr.id]="categoryId(categoria.nombre)">
29     <div class="category-block_head">
30       <p class="label">Categoría</p>
31       <h2>{{ categoria.nombre }}</h2>
32     </div>
33
34     <div class="catalogo-grid">
35       <article class="producto-card" *ngFor="let producto of categoria.productos; trackBy: trackById">
36         <div class="producto-card_media">
37           <img
38             *ngIf="producto.imagenMostrada"
39             [src]="producto.imagenMostrada"
40             [alt]="producto.nombre"
41             (error)="onImageError($event)"
42           />
43         <p class="producto-card_tag">{{ producto.categoria }}</p>
44       </article>
45     </div>
46   </article>
47 </section>
```

```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2
3 import { Productos } from './productos';
4
5 describe('Productos', () => {
6   let component: Productos;
7   let fixture: ComponentFixture<Productos>;
8
9   beforeEach(async () => {
10     await TestBed.configureTestingModule({
11       imports: [Productos]
12     })
13     .compileComponents();
14
15     fixture = TestBed.createComponent(Productos);
16     component = fixture.componentInstance;
17     await fixture.whenStable();
18   });
19
20   it('should create', () => {
21     expect(component).toBeTruthy();
22   });
23 });
```

```
20 })
21 export class Productos implements OnInit {
22   productos: ProductoVista[] = [];
23   categorias: CategoriaGrupo[] = [];
24   cargando = true;
25
26   constructor(
27     private readonly productosService: ProductosService,
28     private readonly promocionesService: PromocionesService,
29     private readonly carrito: Carrito,
30   ) {}
31
32   ngOnInit(): void {
33     forkJoin([
34       productos: this.productosService.getProductos().pipe(catchError(() => of([]))),
35       promociones: this.promocionesService.getPromociones().pipe(catchError(() => of([]))),
36     ]).subscribe(({ productos, promociones }) => {
37       this.productos = mapearProductosConPromociones(productos, promociones);
38       this.categorias = this.agruparPorCategoria(this.productos);
39       this.cargando = false;
40     });
41   }
42
43   trackById(_: number, producto: ProductoVista): string {
44     return producto_id || producto.nombre;
45   }
46
47   agregarAlCarrito(producto: ProductoVista): void {
48     this.carrito.add(producto);
49   }
50
51   onImageError(event: Event): void {
52     const target = event.target as HTMLImageElement | null;
53     if (!target) {
54       return;
55     }
56     target.style.display = 'none';
57   }
58
59   scrollToCategoria(nombre: string): void {
60     if (typeof document === 'undefined') {
61       return;
62     }
63   }
```

Promociones

```
1 <section class="promo-hero">
2   <div>
3     <p class="label">Promociones</p>
4     <h1>Ofertas activas y aplicadas al menú</h1>
5     <p>
6       Aquí ves las promociones que están corriendo ahorita, cuánto bajan el precio y en qué
7       productos aplican.
8     </p>
9   </div>
10 </section>
11
12 <section class="promo-grid">
13   <article class="promo-card" *ngFor="let promo of promociones">
14     <p class="promo-card_type">{{ promo.tipo }}</p>
15     <h2>{{ promo.titulo }}</h2>
16     <p>{{ promo.descripcion }}</p>
17     <strong>{{ promo.etiquetaValor }}</strong>
18     <small *ngIf="promo.productosRelacionados.length">
19       | Aplica a: {{ nombresRelacionados(promo) }}
20     </small>
21     <button type="button" class="promo-card_action" (click)="usarPromocion(promo)">
22       {{ promo.accionLabel }}
23     </button>
24   </article>
25 </section>
26
27 <div class="promo-modal-backdrop" *ngIf="modalSeleccionAbierto" (click)="cerrarSeleccionPromocion()"></div>
28
29 <div class="promo-modal" *ngIf="modalSeleccionAbierto && promocionSeleccionada as promo">
30   <p class="label">Elegir producto:</p>
31   <h2>{{ promo.titulo }}</h2>
32   <p>{{ promo.descripcion }}</p>
33
34   <div class="promo-modal_list">
35     <button type="button" class="promo-modal_item" *ngFor="let producto of promo.productosRelacionados" (click)="elegirProductoPromo(producto)">
36       <div>
37         <strong>{{ producto.nombre }}</strong>
38         <small>{{ producto.descripcion }}</small>
39       </div>
40       <span>{{ producto.precioFinal | number: '1.0-0' }}</span>
41     </button>
42   </div>
43
44   <button type="button" class="promo-modal_close" (click)="cerrarSeleccionPromocion()">Cancelar</button>
```

```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2
3 import { Promociones } from './promociones';
4
5 describe('Promociones', () => {
6   let component: Promociones;
7   let fixture: ComponentFixture<Promociones>;
8
9   beforeEach(async () => {
10     await TestBed.configureTestingModule({
11       imports: [Promociones]
12     })
13     .compileComponents();
14
15     fixture = TestBed.createComponent(Promociones);
16     component = fixture.componentInstance;
17     await fixture.whenStable();
18   });
19
20   it('should create', () => {
21     expect(component).toBeTruthy();
22   });
23
24   it('should have a length of 1', () => {
25     expect(component.promociones.length).toBe(1);
26   });
27 });
```

```
1 import { CommonModule } from '@angular/common';
2 import { Component, OnInit } from '@angular/core';
3 import { catchError, forkJoin, of } from 'rxjs';
4 import { mapToProductosConPromociones, ProductoVista } from '../services/catalogo';
5 import { Carrito } from '../services/carrito';
6 import { Producto, ProductosService } from '../services/productos';
7 import { Promoción, Promociones as PromocionesService } from '../services/promociones';
8
9 type PromocionVista = Promocion & {
10   productosRelacionados: ProductoVista[];
11   etiquetaValor: string;
12   accionLabel: string;
13   agregaDirecto: boolean;
14 };
15
16 @Component({
17   selector: 'app-promociones',
18   standalone: true,
19   imports: [CommonModule],
20   templateUrl: './promociones.html',
21   styleUrls: ['./promociones.css'],
22 })
23 export class Promociones implements OnInit {
24   promociones: PromocionVista[] = [];
25   modalSeleccionAbierto = false;
26   promocionSeleccionada: PromocionVista | null = null;
27
28   constructor(
29     private readonly productosService: ProductosService,
30     private readonly promocionesService: PromocionesService,
31     private readonly carrito: Carrito,
32   ) {}
33
34   ngOnInit(): void {
35     forkJoin([
36       productos: this.productosService.getProductos().pipe(catchError(() => of([]))),
37       promociones: this.promocionesService.getPromociones().pipe(catchError(() => of([]))),
38     ]).subscribe(({ productos, promociones }) => {
39       const productosVista = mapToProductosConPromociones(productos, promociones);
40       this.promociones = promociones.map((promocion) => ({
41         ...promocion,
42         productosRelacionados: productosVista.filter((producto) =>
43           !promocion.producto_ids || !promocion.producto_ids.includes(producto.id || ''),
44         ));
45     });
46   }
47 }
```

Services

This screenshot shows the VS Code editor with the file explorer on the left and the editor window on the right. The file explorer shows the project structure for 'PROYECTO_WEB_EQUIPOKJHUF', with the 'admin.guard.ts' file selected under the 'services' directory. The editor window displays the code for 'admin.guard.ts', which implements the 'CanActivate' guard. The code includes imports for 'Injectable', 'CanActivate', 'Router', and 'Auth' from '@angular/core', '@angular/router', and './auth' respectively. It defines an '@Injectable' decorator with 'providedIn: root' and a 'constructor' that takes 'AuthService' and 'Router' as parameters. The 'canActivate()' method checks if the user is authenticated; if not, it navigates to '/admin/login' and returns false, otherwise, it returns true.

```
1 import { Injectable } from '@angular/core';
2 import { CanActivate, Router } from '@angular/router';
3 import { Auth } from './auth';
4
5 @Injectable({
6   providedIn: 'root'
7 })
8 export class AdminGuard implements CanActivate {
9
10  constructor(
11    private authService: Auth,
12    private router: Router
13  ) {}
14
15  canActivate(): boolean {
16    if (this.authService.isAuthenticated()) {
17      return true;
18    } else {
19      this.router.navigate(['/admin/login']);
20      return false;
21    }
22  }
23 }
24
```

This screenshot shows the VS Code editor with the file explorer on the left and the editor window on the right. The file explorer shows the project structure for 'PROYECTO_WEB_EQUIPOKJHUF', with the 'auth.spec.ts' file selected under the 'services' directory. The editor window displays the code for 'auth.spec.ts', which contains unit tests for the 'Auth' service. The code includes imports for 'TestBed' from '@angular/core/testing' and 'Auth' from './auth'. It defines a 'describe' block for 'Auth' with a 'beforeEach' hook that configures the testing module and injects the 'Auth' service. The 'it' block tests that the service is created and returns a truthy value.

```
1 import { TestBed } from '@angular/core/testing';
2
3 import { Auth } from './auth';
4
5 describe('Auth', () => {
6   let service: Auth;
7
8   beforeEach(() => {
9     TestBed.configureTestingModule({});
10    service = TestBed.inject(Auth);
11  });
12
13   it('should be created', () => {
14     expect(service).toBeTruthy();
15   });
16 });
17
```

File Edit Selection View Go Run Terminal Help

Projecto.Web_EquipoKJHUF

Update

EXPLORER

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productots.ts

cliente

services

admin.guard.ts

auth.spec.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spec.ts

modals.ts

pedidos.ts

preload.ts

productots.spec.ts

productots

promociones.spec.ts

promociones.ts

seed-data.ts

usuarios.ts

shared

cart

footer

header

navbar

OUTLINE

TIMELINE

TS navbar.html

TS navbar.ts

home.html

TS auth.ts

k-jhuf-web > src > app > services > TS auth.ts > Auth

```
1 import { HttpClient, HttpHeaders } from '@angular/common/http';
2 import { Injectable } from '@angular/core';
3 import { Observable, tap } from 'rxjs';
4 import { environment } from '../../environment';
5
6 type AuthResponse = {
7   msg: string;
8   token: string;
9 };
10
11 type AdminStatusResponse = {
12   configured: boolean;
13 };
14
15 @Injectable({
16   providedIn: 'root',
17 })
18 export class Auth {
19   private readonly api = `${environment.apiUrl}/api/admin`;
20   private readonly tokenKey = 'kjhuf admin token';
21
22   constructor(private readonly http: HttpClient) {}
23
24   status(): Observable<AdminStatusResponse> {
25     return this.http.get<AdminStatusResponse>(`${this.api}/status`);
26   }
27
28   setup(password: string): Observable<AuthResponse> {
29     return this.http.post<AuthResponse>(`${this.api}/setup`, { password }).pipe(
30       tap((response) => this.saveToken(response.token)),
31     );
32   }
33
34   login(password: string): Observable<AuthResponse> {
35     return this.http.post<AuthResponse>(`${this.api}/login`, { password }).pipe(
36       tap((response) => this.saveToken(response.token)),
37     );
38   }
39
40   logout(): Observable<{ msg: string }> {
41     return this.http
42       .post<{ msg: string }>(`${this.api}/logout`, {}, { headers: this.getAuthHeaders() })
43       .pipe(tap(() => this.clearToken()));
```

File Edit Selection View Go Run Terminal Help

Projecto.Web_EquipoKJHUF

Update

EXPLORER

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productots.ts

cliente

services

admin.guard.ts

auth.spec.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spec.ts

modals.ts

pedidos.ts

preload.ts

productots.spec.ts

productots

promociones.spec.ts

promociones.ts

seed-data.ts

usuarios.ts

shared

cart

footer

header

navbar

OUTLINE

TIMELINE

TS navbar.html

TS navbar.ts

home.html

TS carrito.ts

k-jhuf-web > src > app > services > TS carrito.ts > ...

```
1 import { Injectable } from '@angular/core';
2 import { BehaviorSubject } from 'rxjs';
3 import { ProductoVista } from './catalogo';
4 import { Promocion } from './promociones';
5
6 export type CartItem = {
7   producto_id: string;
8   nombre: string;
9   precio: number;
10   precioOriginal: number;
11   cantidad: number;
12   imagen_url: string;
13   promoResumen: string;
14   promocionesAplicadas: Promocion[];
15 };
16
17 export type CartDiscount = {
18   label: string;
19   amount: number;
20 };
21
22 export type CartPricing = {
23   subtotal: number;
24   descuentos: CartDiscount[];
25   total: number;
26 };
27
28 @Injectable({
29   providedIn: 'root',
30 })
31 export class Carrito {
32   private readonly storageKey = 'kjhuf_cart';
33   private readonly abierto$ = new BehaviorSubject<boolean>(false);
34   private readonly items$ = new BehaviorSubject<CartItem[]>(this.readItems());
35
36   readonly abierto = this.abierto$.asObservable();
37   readonly items = this.items$.asObservable();
38
39   open(): void {
40     this.abierto$.next(true);
41   }
42
43   close(): void {
```

```
1 import { Producto } from './productos';
2 import { Promocion } from './promociones';
3
4 export type ProductoVista = Producto & {
5     imagenMostrada: string;
6     promocionesAplicadas: Promocion[];
7     precioFinal: number;
8     resumenPromo: string;
9 };
10
11 export function mapearProductosConPromociones(
12     productos: Producto[],
13     promociones: Promocion[],
14 ): ProductoVista[] {
15     return productos.map((producto) => {
16         const promocionesAplicadas = promociones.filter((promocion) => {
17             const ids = promocion.producto_ids || [];
18             return promocion.activo !== false && ids.includes(producto_id || '');
19         });
20
21         return {
22             ...producto,
23             ingredientes: producto.ingredientes || [],
24             detalles: producto.detalles || [],
25             destacado: !!producto.destacado,
26             imagenMostrada: (producto.imagen_url || producto.imagen || '').trim(),
27             promocionesAplicadas,
28             precioFinal: calcularPrecioPromocional(producto.precio, promocionesAplicadas),
29             resumenPromo: obtenerResumenPromo(promocionesAplicadas),
30         };
31     });
32 }
33
34 export function calcularPrecioPromocional(precio: number, promociones: Promocion[]): number {
```

```
1 import { HttpClient } from '@angular/common/http';
2 import { Injectable } from '@angular/core';
3 import { Observable, shareReplay, tap } from 'rxjs';
4 import { environment } from '../environment';
5 import { Auth } from './auth';
6
7 export type HomeConfig = {
8     home_banner_url: string;
9     store_address: string;
10     store_phone: string;
11     store_hours: string;
12     store_maps_url: string;
13 };
14
15 @Injectable({
16     providedIn: 'root',
17 })
18 export class HomeConfigService {
19     private readonly publicApi = `${environment.apiUrl}/api/home-config`;
20     private readonly adminApi = `${environment.apiUrl}/api/admin/home-config`;
21     private readonly imageProxyApi = `${environment.apiUrl}/api/imagen-proxy`;
22     private config$: Observable<HomeConfig>;
23
24     constructor(
25         private readonly http: HttpClient,
26         private readonly auth: Auth,
27     ) {}
28
29     getConfig(forceRefresh = false): Observable<HomeConfig> {
30         if (forceRefresh || !this.config$) {
31             this.config$ = this.http.get<HomeConfig>(this.publicApi).pipe(shareReplay(1));
32         }
33
34         return this.config$;
35     }
36
37     updateConfig(payload: Partial<HomeConfig>): Observable<HomeConfig & { msg: string }> {
38         return this.http.patch<HomeConfig & { msg: string }>(
39             this.adminApi,
40             payload,
41             { headers: this.auth.getAuthHeaders() },
42         ).pipe(tap(() => this.invalidateCache()));
43     }
44 }
```


File Edit Selection View Go Run Terminal Help

Proyecto_Web_EquipokJHUF

Update

EXPLORER

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productos.ts

cliente

services

admin.guard.ts

auth.spect.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spect.ts

modal.ts

pedidos.ts

preload.ts

productos.spect.ts

productos.ts

promociones.spect.ts

TS modal.service.ts

```
1 import { Injectable } from '@angular/core';
2 import { BehaviorSubject, Observable } from 'rxjs';
3
4 @Injectable({
5   providedIn: 'root'
6 })
7 export class ModalService {
8   private loginModalVisibleSubject = new BehaviorSubject<boolean>(false);
9   loginModalVisible$: Observable<boolean> = this.loginModalVisibleSubject.asObservable();
10
11   constructor() {}
12
13   openLoginModal(): void {
14     this.loginModalVisibleSubject.next(true);
15   }
16
17   closeLoginModal(): void {
18     this.loginModalVisibleSubject.next(false);
19   }
20 }
21
```

File Edit Selection View Go Run Terminal Help

Proyecto_Web_EquipokJHUF

Update

EXPLORER

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productos.ts

cliente

services

admin.guard.ts

auth.spect.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spect.ts

modal.ts

pedidos.ts

TS modal.spect.ts

```
1 import { TestBed } from '@angular/core/testing';
2 import { ModalService } from './modal.service';
3
4 describe('ModalService', () => {
5   let service: ModalService;
6
7   beforeEach(() => {
8     TestBed.configureTestingModule({
9       service: TestBed.inject(ModalService);
10     });
11
12     it('should be created', () => {
13       expect(service).toBeTruthy();
14     });
15
16     it('should open and close login modal', (done) => {
17       service.loginModalVisible$.subscribe(visible => {
18         expect(visible).toBeDefined();
19       });
20
21       service.openLoginModal();
22       service.closeLoginModal();
23       done();
24     });
25   });
26 }
```

File Edit Selection View Go Run Terminal Help

ProyectoWeb_EquipoKJHUF

Update

EXPLORES

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productos.ts

cliente

services

admin.guard.ts

auth.spect.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spect.ts

modal.ts

pedidos.ts

preload.ts

productos.spect.ts

productos.ts

promociones.spect.ts

promociones.ts

seed-data.ts

usuarios.ts

shared

cart

footer

header

navbar

OUTLINE

TIMELINE

home 01 201 Launchpad 2 0

Ln 1, Col 1 Spaces: 2 UTF-8 CRLF TypeScript Go Live Prettier

```
1 import { HttpClient } from '@angular/common/http';
2 import { Injectable } from '@angular/core';
3 import { Observable, timeout } from 'rxjs';
4 import { environment } from '../environment';
5 import { Auth } from './auth';
6 import { UsuariosService } from './usuarios';
7
8 export type PedidoItem = {
9   producto_id: string;
10  nombre: string;
11  precio: number;
12  precio_original?: number;
13  cantidad: number;
14  imagen_url: string;
15  subtotal?: number;
16 };
17
18 export type PedidoDescuento = {
19   titulo: string;
20   tipo: string;
21   monto: number;
22 };
23
24 export type Pedido = {
25   _id?: string;
26   cliente: string;
27   telefono: string;
28   notas: string;
29   hora_entrega?: string;
30   fecha_pedido_local?: string;
31   estado: 'pendiente' | 'en_proceso' | 'entregado' | 'cancelado';
32   items: PedidoItem[];
33   subtotal?: number;
34   descuentos_aplicados?: PedidoDescuento[];
35   total: number;
36   creado_en?: string;
37 };
38
39 export type PedidoPendiente = Omit<Pedido, 'estado'> & {
40   estado: 'pendiente';
41   pendiente_local?: boolean;
42 };
43
44 export type DashboardAdmin = {
```

File Edit Selection View Go Run Terminal Help

ProyectoWeb_EquipoKJHUF

Update

EXPLORES

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

node_modules

public

scripts

src

app

admin\productos

productos.css

productos.html

productos.ts

cliente

services

admin.guard.ts

auth.spect.ts

auth.ts

carrito.ts

catalogo.ts

home-config.ts

modal.service.ts

modal.spect.ts

modal.ts

pedidos.ts

preload.ts

productos.spect.ts

productos.ts

promociones.spect.ts

promociones.ts

seed-data.ts

usuarios.ts

shared

cart

footer

header

navbar

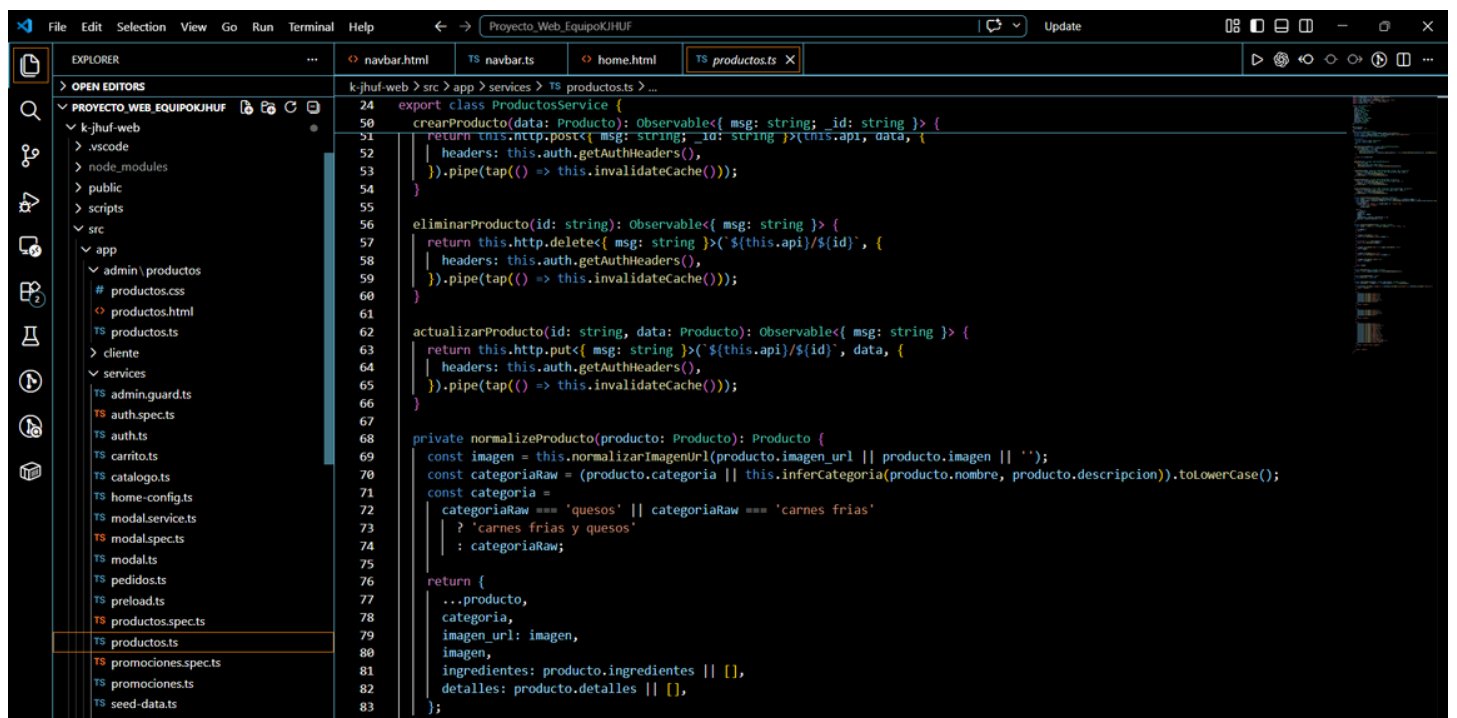
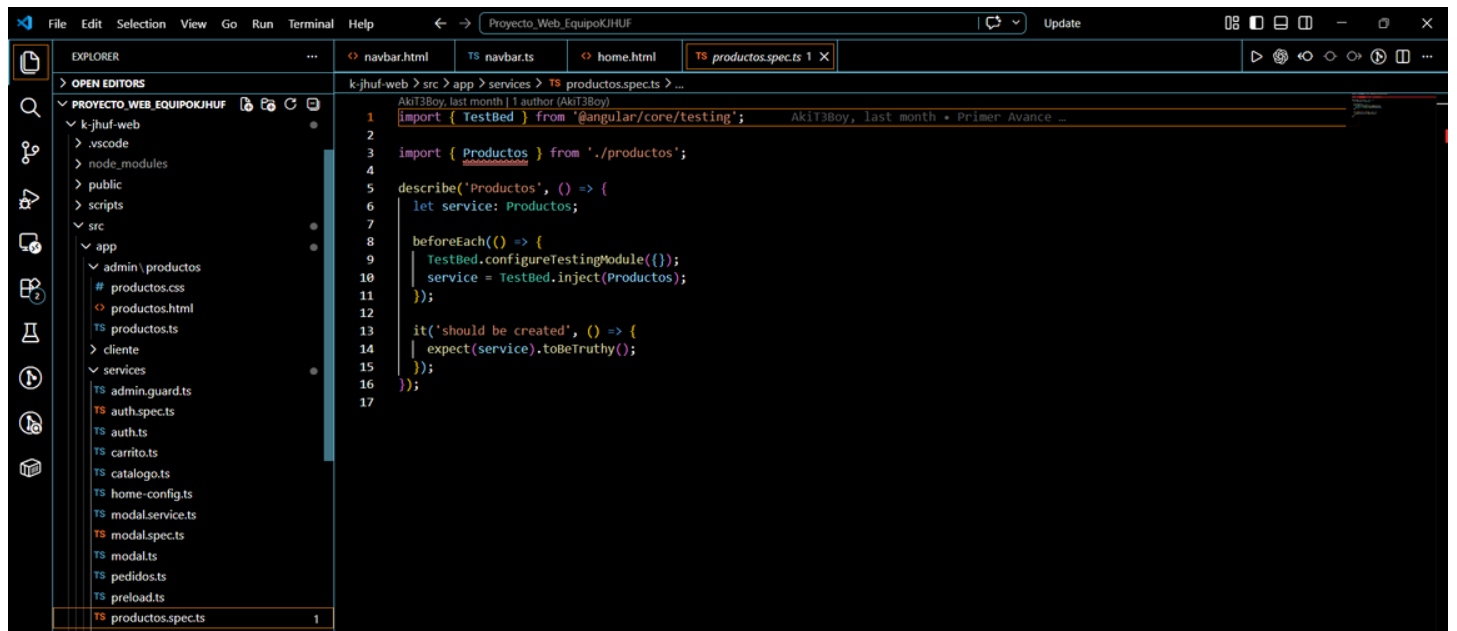
OUTLINE

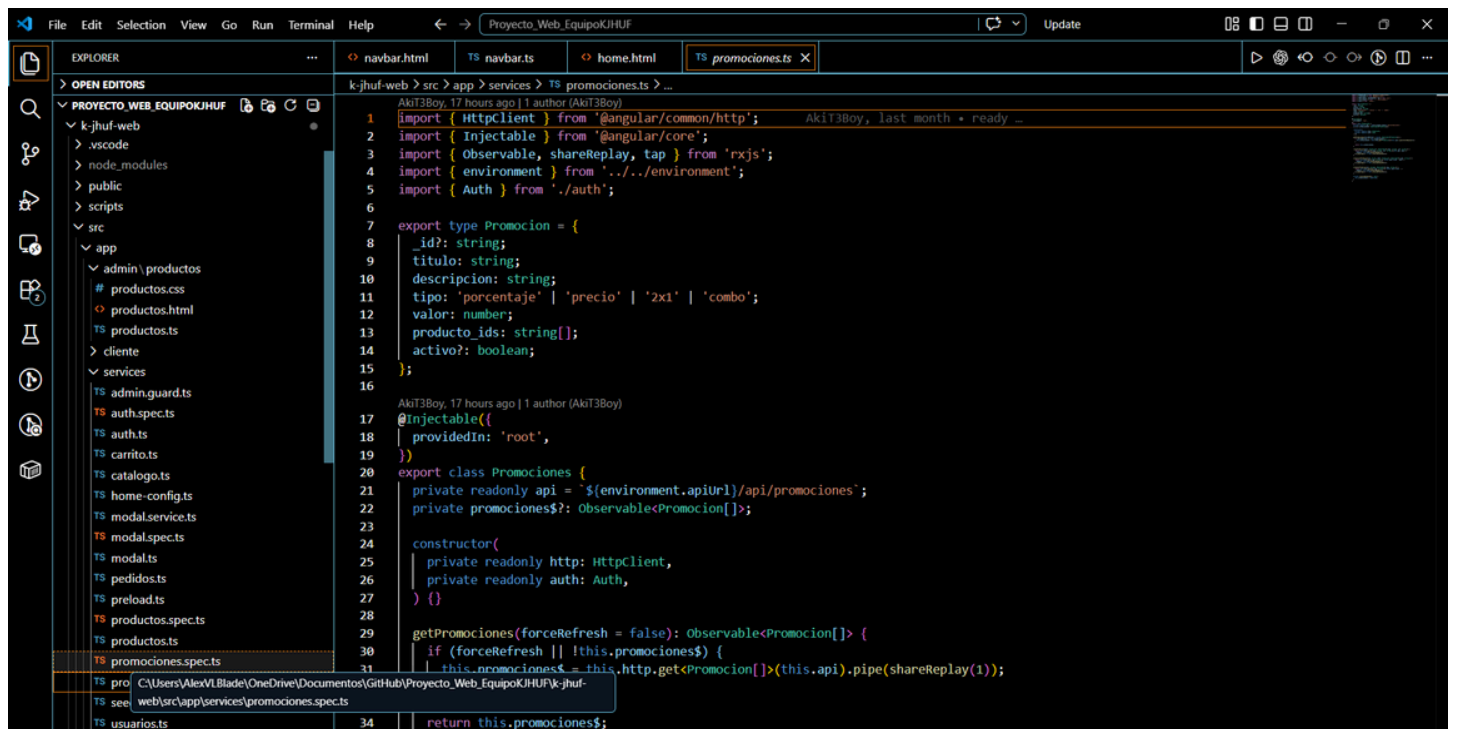
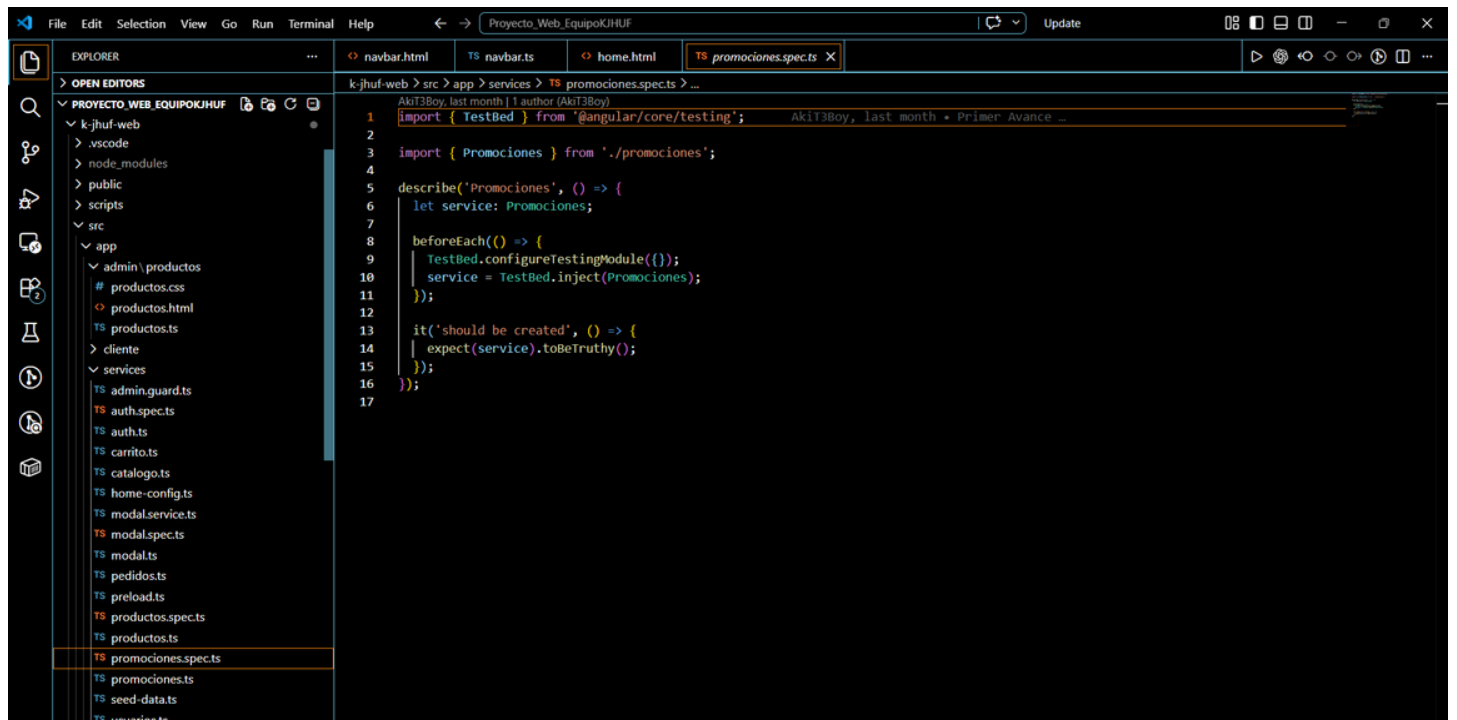
TIMELINE

home 01 201 Launchpad 2 0

Ln 1, Col 1 Spaces: 2 UTF-8 CRLF TypeScript Go Live Prettier

```
1 import { Injectable } from '@angular/core';
2 import { forkJoin, of } from 'rxjs';
3 import { catchError } from 'rxjs/operators';
4 import { HomeConfigService } from './home-config';
5 import { Pedidos } from './pedidos';
6 import { ProductosService } from './productos';
7 import { Promociones } from './promociones';
8 import { UsuariosService } from './usuarios';
9
10 @Injectable({
11   providedIn: 'root',
12 })
13 export class PreloadService {
14   private basePreloaded = false;
15   private userPreloadedForToken = '';
16
17   constructor(
18     private readonly productos: ProductosService,
19     private readonly promociones: Promociones,
20     private readonly homeConfig: HomeConfigService,
21     private readonly usuarios: UsuariosService,
22     private readonly pedidos: Pedidos,
23   ) {}
24
25   preloadBase(): void {
26     if (this.basePreloaded) {
27       return;
28     }
29
30     this.basePreloaded = true;
31     forkJoin([
32       productos: this.productos.getProductos().pipe(catchError(() => of([]))),
33       promociones: this.promociones.getPromociones().pipe(catchError(() => of([]))),
34       home: this.homeConfig.getConfig().pipe(
35         catchError(() => of({
36           home_banner_url: '',
37           store_address: '',
38           store_phone: '',
39           store_hours: '',
40           store_maps_url: '',
41         })),
42       ),
43     ]),
```





The screenshot shows the VS Code editor interface with the Explorer sidebar on the left displaying the project structure. The main editor area shows the file `seed-data.ts` in the `src > app > services` directory. The code defines an array of product seed data for a REST API.

```
1 import { Producto } from './productos';
2 import { Promocion } from './promociones';
3
4 export const PRODUCTOS_SEED: Producto[] = [
5   {
6     _id: 'seed-esquites-chico',
7     nombre: 'Esquites Chico',
8     descripcion: 'Esquites chicos cremosos con maiz tierno, limon y toque de chile.',
9     precio: 40,
10    categoria: 'elotes',
11    imagen_url: '',
12    ingredientes: ['Maiz desgranado', 'Mayonesa', 'Queso', 'Chile', 'Limon'],
13    detalles: ['Preparado al momento', 'Picante ajustable'],
14    destacado: true,
15    activo: true,
16  },
17  {
18    _id: 'seed-esquites-mediano',
19    nombre: 'Esquites Mediano',
20    descripcion: 'Porcion mediana de esquites con mas queso y mas limon.',
21    precio: 45,
22    categoria: 'elotes',
23    imagen_url: '',
24    ingredientes: ['Maiz desgranado', 'Mayonesa', 'Queso extra', 'Chile', 'Limon'],
25    detalles: ['Ideal para antojo personal', 'Salsa al gusto'],
26    destacado: false,
27    activo: true,
28  },
29  {
30    _id: 'seed-maruchan-locas',
31    nombre: 'Maruchan Loca',
32    descripcion: 'Maruchan preparada con cueritos, queso, salsa y limon.',
33    precio: 80,
34    categoria: 'snacks',
35    imagen_url: '',
36    ingredientes: ['Maruchan', 'Queso', 'Cueritos', 'Salsa', 'Limon'],
37    detalles: ['Calientita al servir', 'Botana completa'],
38    destacado: true,
39    activo: true,
40  },
41  {
42    _id: 'seed-nachos-especiales',
43    nombre: 'Nachos Especiales',
44    descripcion: 'Tortitas crujientes con queso, jalapeno y aderezo.'
45  }
46 ]
```

The screenshot shows the VS Code editor interface with the Explorer sidebar on the left displaying the project structure. The main editor area shows the file `usuarios.ts` in the `src > app > services` directory. The code defines the `UsuarioSession` and `UsuarioAuthService` classes.

```
1 import { HttpClient, HttpHeaders } from '@angular/common/http';
2 import { Injectable } from '@angular/core';
3 import { BehaviorSubject, Observable, tap } from 'rxjs';
4 import { environment } from '../environment';
5
6 export type UsuarioSession = {
7   _id?: string;
8   nombre: string;
9   telefono: string;
10  creado_en?: string;
11 };
12
13 type UsuarioAuthResponse = {
14   msg: string;
15   token: string;
16   usuario: UsuarioSession;
17 };
18
19 @Injectable({
20   providedIn: 'root',
21 })
22 export class UsuariosService {
23   private readonly api = `${environment.apiUrl}/api/usuarios`;
24   private readonly tokenKey = 'kj_huf_user_token';
25   private readonly usuarioSubject = new BehaviorSubject<UsuarioSession | null>(this.readStoredUser());
26
27   readonly usuario$ = this.usuarioSubject.asObservable();
28
29   constructor(private readonly http: HttpClient) {}
30
31   register(payload: { nombre: string; telefono: string; password: string }): Observable<UsuarioAuthResponse> {
32     return this.http.post<UsuarioAuthResponse>(`${this.api}/register`, payload).pipe(
33       tap((response) => this.saveSession(response.token, response.usuario)),
34     );
35   }
36
37   login(payload: { telefono: string; password: string }): Observable<UsuarioAuthResponse> {
38     return this.http.post<UsuarioAuthResponse>(`${this.api}/login`, payload).pipe(
39       tap((response) => this.saveSession(response.token, response.usuario)),
40     );
41   }
42
43   me(): Observable<UsuarioSession> {
```

Shared

App.py

```
1  AkiT3Boy, 14 hours ago | 2 authors (AkiT3Boy and one other)
2  import os
3  from functools import wraps
4  import json
5  from secrets import token_hex
6  from datetime import datetime, timezone, timedelta
7  from zoneinfo import ZoneInfo, ZoneInfoNotFoundError
8  from urllib.parse import urlparse
9  from urllib.request import Request, urlopen
10
11 from bson import ObjectId
12 from bson.errors import InvalidId
13 from flask import Flask, Response, jsonify, request, g
14 from flask_cors import CORS
15 from flask_pymongo import PyMongo
16 from pymongo.errors import DuplicateKeyError, PyMongoError
17 from werkzeug.security import check_password_hash, generate_password_hash
18
19 app = Flask(__name__)
20 CORS(app)
21
22 app.config["MONGO_URI"] = os.getenv("MONGO_URI", "mongodb://localhost:27017/kjhuf")
23 mongo = PyMongo(app)
24
25 ADMIN_TOKENS = set()
26 USER_TOKENS = set()
27
28 try:
29     LOCAL_TZ = ZoneInfo("America/Mexico_City")
30 except ZoneInfoNotFoundError:
31     LOCAL_TZ = timezone(timedelta(hours=-6))
32
33 BUSINESS_OPEN_MINUTES = 18 * 60
34 BUSINESS_CLOSE_MINUTES = 23 * 60
35
36 DEFAULT_STORE_ADDRESS = "616 Adolfo Lopez Mateos, Poza Rica, Veracruz"
37 DEFAULT_STORE_PHONE = "7822174525"
38
39 DEFAULT_STORE_HOURS = "Lunes a domingo, 5:00 p. m. - 11:00 p. m."
40
41 DEFAULT_STORE_MAPS_URL = (
42     "https://www.google.com/maps?q=616+Adolfo+Lopez+Mateos+Poza+Rica,+Veracruz&z=16&output=embed"
43 )
44
45 def parse_object_id(value):
46     try:
47         return ObjectId(value)
48     except (InvalidId, TypeError):
49         return None
```

Navbar

File Edit Selection View Go Run Terminal Help

Proyecto.Web_EquipoKJHUF

Update

nav.html TS navbar.ts home.html TS productos.ts

EXPLORES

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

src

app

admin

cliente

services

shared

cart

footer

header

navbar

navbar.css

navbar.html

navbar.spec.ts

navbar.ts

app.config.server.ts

app.config.ts

app.css

app.html

app.routes.server.ts

app.routes.ts

app.spec.ts

app.ts

assets

icons

images

environment.prod.template.ts

environment.prod.ts

environment.ts

index.html

main.server.ts

OUTLINE

TIMELINE

3 <div class="sidebar_auth-modal" *ngIf="modalUsuarioAbierto">

4 <div class="sidebar_auth-dialog">

14 <label *ngIf="modoUsuario === 'registro'">

16 <input

19 (blur)="marcarAuthTouched('nombre')"

20 name="sidebarAuthNombre"

21 maxlength="25"

22 minlength="3"

23 type="text"

24 class="form-control"

25 pattern="^[A-Za-zÁÉÍÓÚáéíóúñ]+\$"

26 autocomplete="name"

27 (keypress)="soloLetras(\$event)"

28 />

29 <small class="sidebar_field-error" *ngIf="authNombreError">{{ authNombreError }}</small>

30 <small class="sidebar_field-warning" *ngIf="!authNombreError && authNombre.length === 25">

31 Se alcanzó el límite de 25 caracteres.

32 </small>

33 </label>

34

35 <label>

36 Teléfono

37 <input

38 [ngModel]="authTelefono"

39 (ngModelChange)="onAuthTelefonoChange(\$event)"

40 (blur)="marcarAuthTouched('telefono')"

41 name="sidebarAuthTelefono"

42 type="tel"

43 class="form-control"

44 maxlength="10"

45 inputmode="numeric"

46 autocomplete="tel"

47 (keypress)="soloNumeros(\$event)"

48 />

49 <small class="sidebar_field-error" *ngIf="authTelefonoError">{{ authTelefonoError }}</small>

50 </label>

51

52 <label>

53 Contraseña

54 <input

55 [ngModel]="authPassword"

56 (ngModelChange)="onAuthPasswordChange(\$event)"

57 (blur)="marcarAuthTouched('password')"

58 name="sidebarAuthPassword"

estebanbaca-code (1 day ago) Ln 179, Col 9 Spaces: 2 UTF-8 CRLF HTML Go Live Prettier

File Edit Selection View Go Run Terminal Help

Proyecto.Web_EquipoKJHUF

Update

nav.html TS navbar.ts home.html TS productos.ts

EXPLORES

OPEN EDITORS

PROYECTO_WEB_EQUIPOKJHUF

k-jhuf-web

src

app

admin

cliente

services

shared

cart

footer

header

navbar

navbar.css

navbar.html

navbar.spec.ts

navbar.ts

app.config.server.ts

app.config.ts

app.css

app.html

app.routes.server.ts

app.routes.ts

app.spec.ts

app.ts

assets

icons

images

environment.prod.template.ts

environment.prod.ts

environment.ts

index.html

main.server.ts

OUTLINE

TIMELINE

3 <div class="sidebar_auth-modal" *ngIf="modalUsuarioAbierto">

4 <div class="sidebar_auth-dialog">

14 <label *ngIf="modoUsuario === 'registro'">

16 <input

19 (blur)="marcarAuthTouched('nombre')"

20 name="sidebarAuthNombre"

21 maxlength="25"

22 minlength="3"

23 type="text"

24 class="form-control"

25 pattern="^[A-Za-zÁÉÍÓÚáéíóúñ]+\$"

26 autocomplete="name"

27 (keypress)="soloLetras(\$event)"

28 />

29 <small class="sidebar_field-error" *ngIf="authNombreError">{{ authNombreError }}</small>

30 <small class="sidebar_field-warning" *ngIf="!authNombreError && authNombre.length === 25">

31 Se alcanzó el límite de 25 caracteres.

32 </small>

33 </label>

34

35 <label>

36 Teléfono

37 <input

38 [ngModel]="authTelefono"

39 (ngModelChange)="onAuthTelefonoChange(\$event)"

40 (blur)="marcarAuthTouched('telefono')"

41 name="sidebarAuthTelefono"

42 type="tel"

43 class="form-control"

44 maxlength="10"

45 inputmode="numeric"

46 autocomplete="tel"

47 (keypress)="soloNumeros(\$event)"

48 />

49 <small class="sidebar_field-error" *ngIf="authTelefonoError">{{ authTelefonoError }}</small>

50 </label>

51

52 <label>

53 Contraseña

54 <input

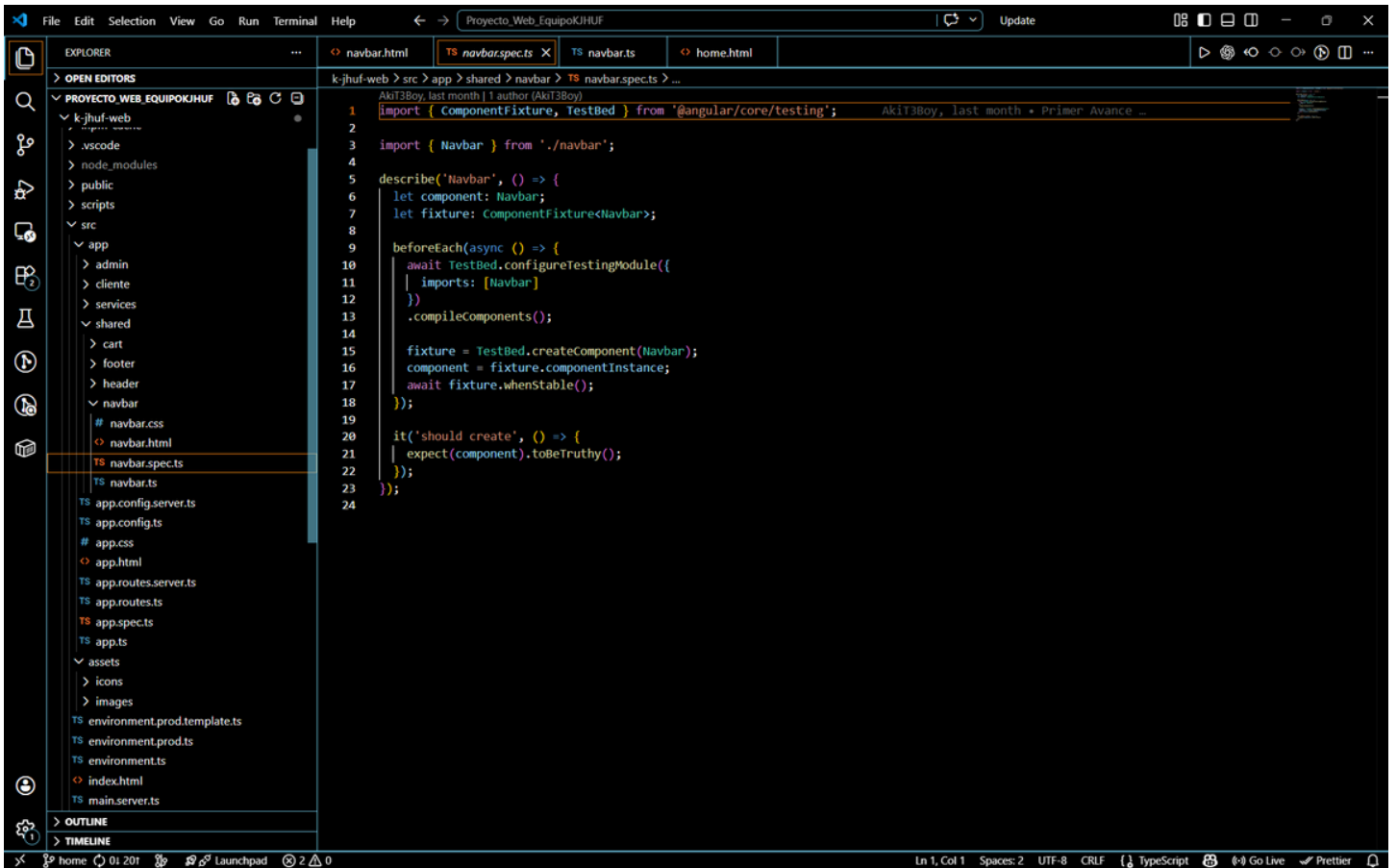
55 [ngModel]="authPassword"

56 (ngModelChange)="onAuthPasswordChange(\$event)"

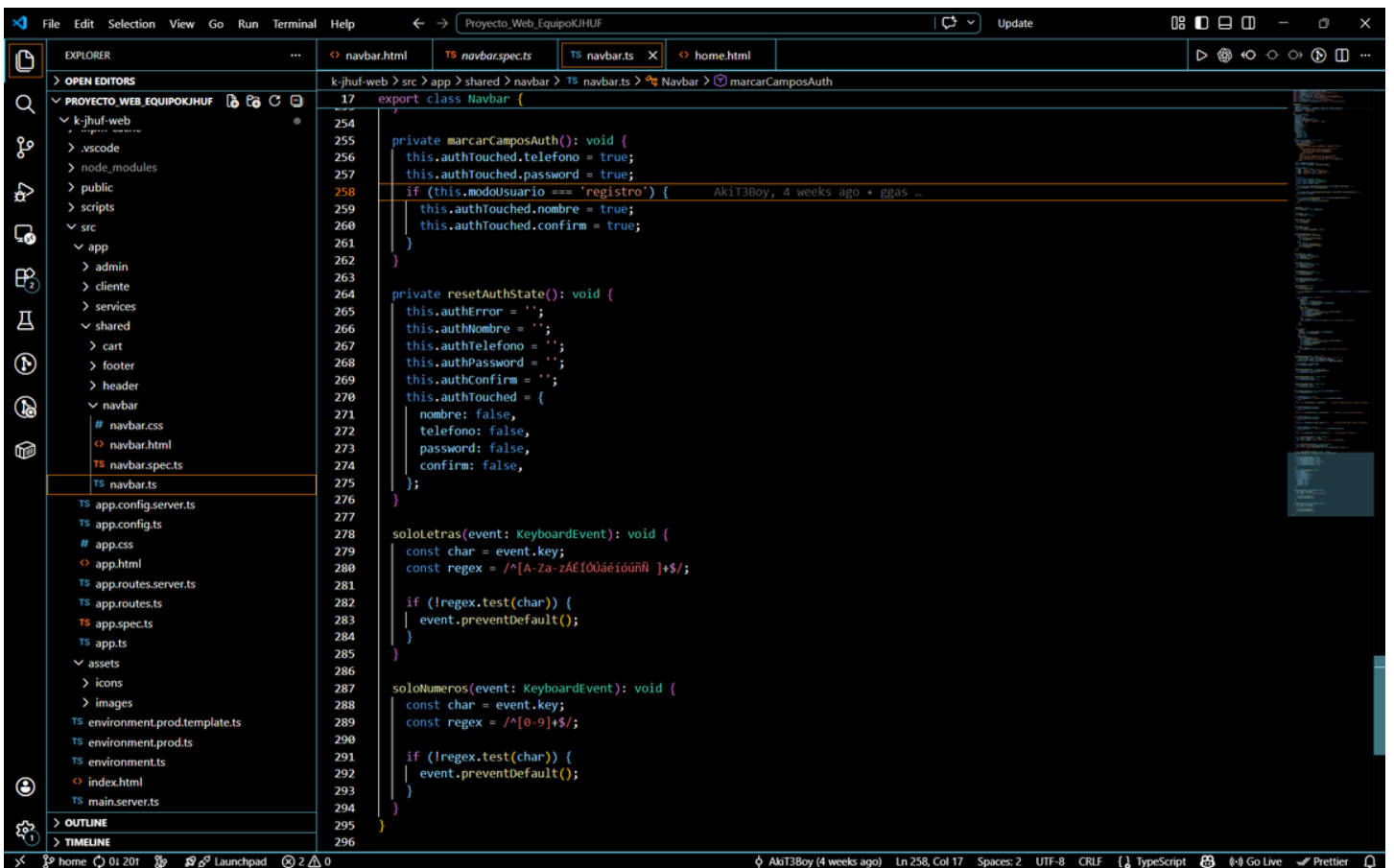
57 (blur)="marcarAuthTouched('password')"

58 name="sidebarAuthPassword"

estebanbaca-code (1 day ago) Ln 179, Col 9 Spaces: 2 UTF-8 CRLF HTML Go Live Prettier



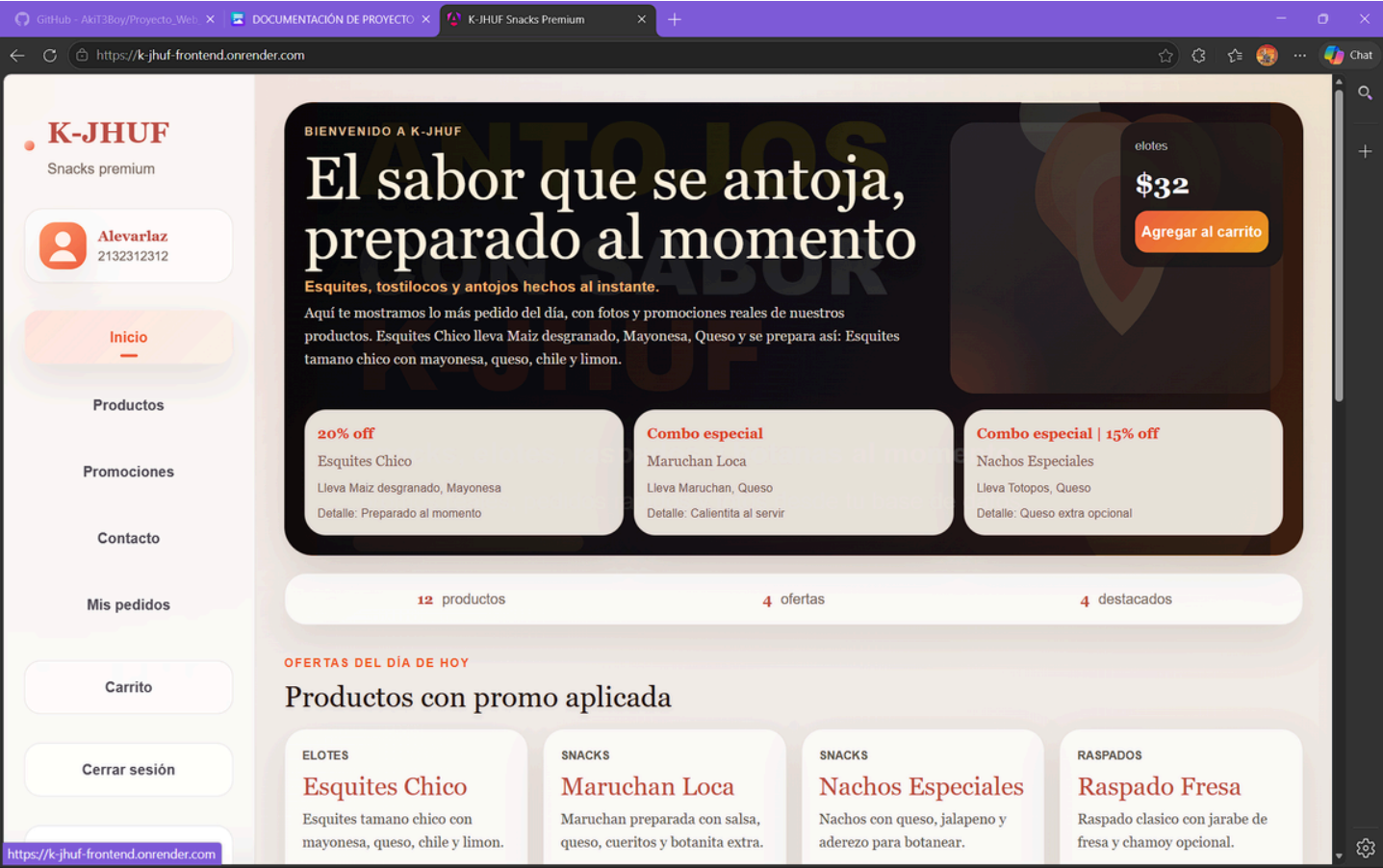
```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2
3 import { Navbar } from './navbar';
4
5 describe('Navbar', () => {
6   let component: Navbar;
7   let fixture: ComponentFixture<Navbar>;
8
9   beforeEach(async () => {
10     await TestBed.configureTestingModule({
11       imports: [Navbar]
12     })
13     .compileComponents();
14
15     fixture = TestBed.createComponent(Navbar);
16     component = fixture.componentInstance;
17     await fixture.whenStable();
18   });
19
20   it('should create', () => {
21     expect(component).toBeTruthy();
22   });
23 });
24
```



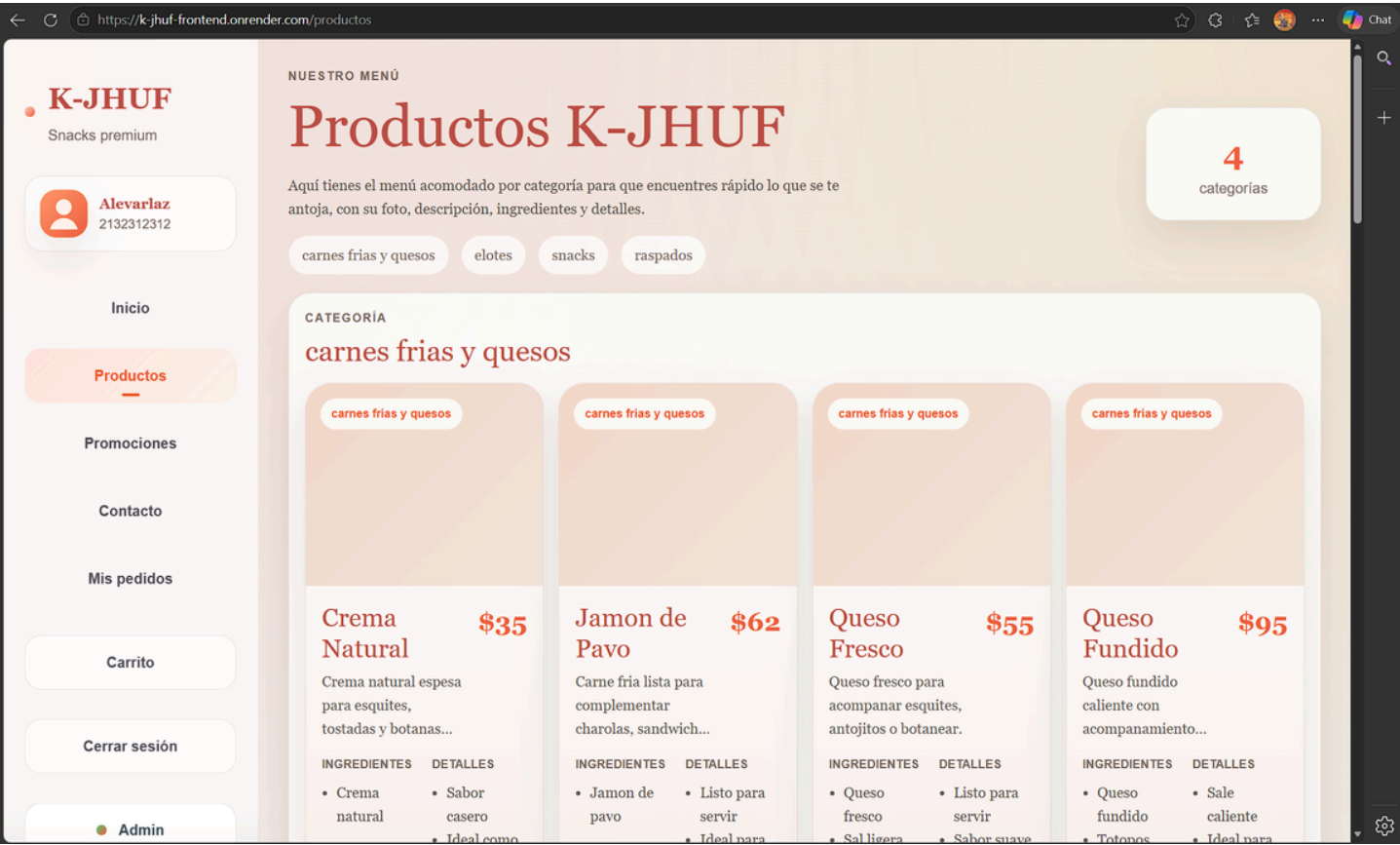
```
17 export class Navbar {
18
19   private marcarCamposAuth(): void {
20     this.authTouched.telefono = true;
21     this.authTouched.password = true;
22     if (this.modouuario === 'registro') {
23       this.authTouched.nombre = true;
24       this.authTouched.confirm = true;
25     }
26   }
27
28   private resetAuthState(): void {
29     this.authError = '';
30     this.authNombre = '';
31     this.authTelefono = '';
32     this.authPassword = '';
33     this.authConfirm = '';
34     this.authTouched = {
35       nombre: false,
36       telefono: false,
37       password: false,
38       confirm: false,
39     };
40   }
41
42   soloLetras(event: KeyboardEvent): void {
43     const char = event.key;
44     const regex = /^[A-Za-zÀÉÍÓÚáéíóúñ ]+$/;
45
46     if (!regex.test(char)) {
47       event.preventDefault();
48     }
49   }
50
51   soloNumeros(event: KeyboardEvent): void {
52     const char = event.key;
53     const regex = /^[0-9]+$/;
54
55     if (!regex.test(char)) {
56       event.preventDefault();
57     }
58   }
59 }
60
```

Proyecto finalizado

Inicio:



Productos:



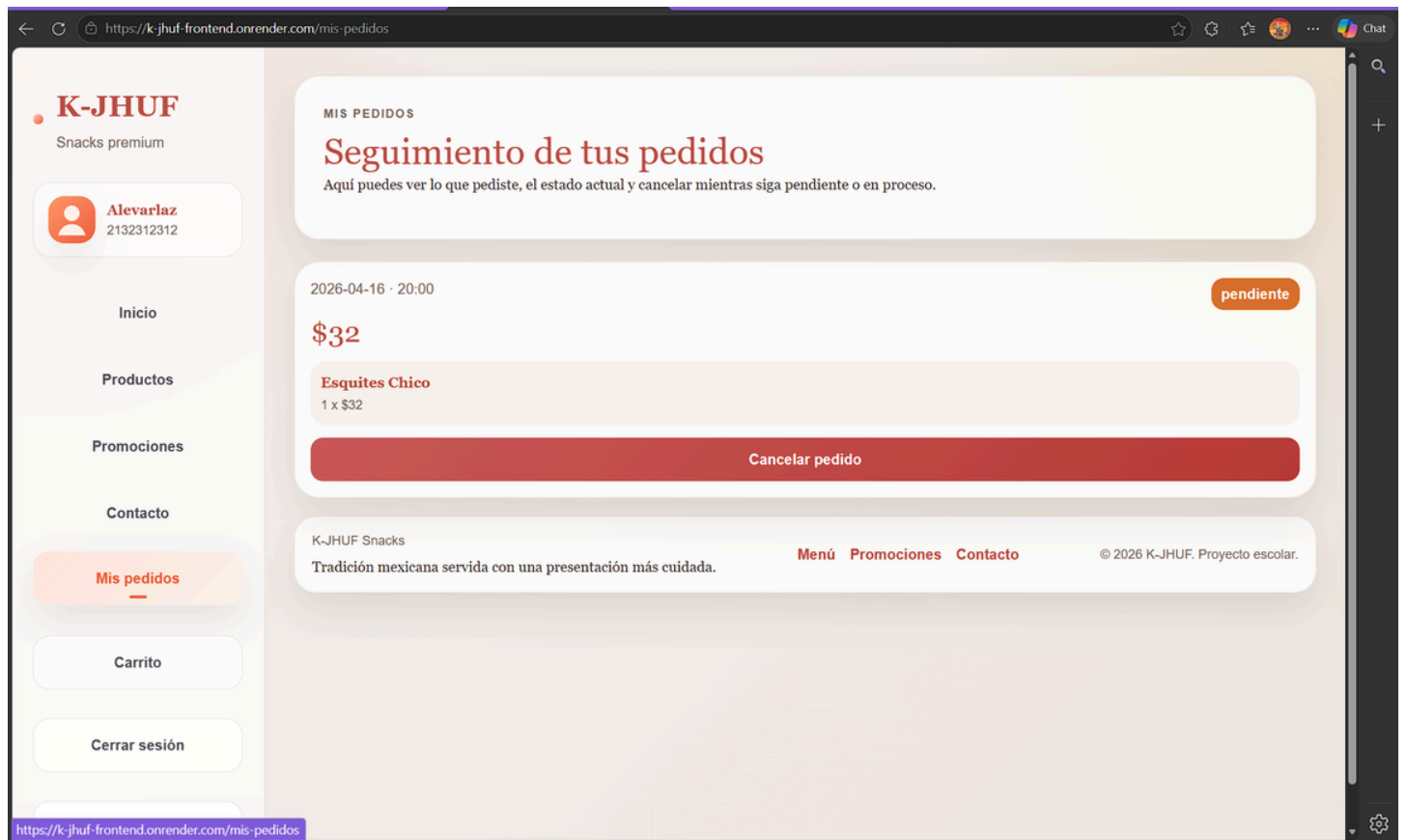
Promociones:



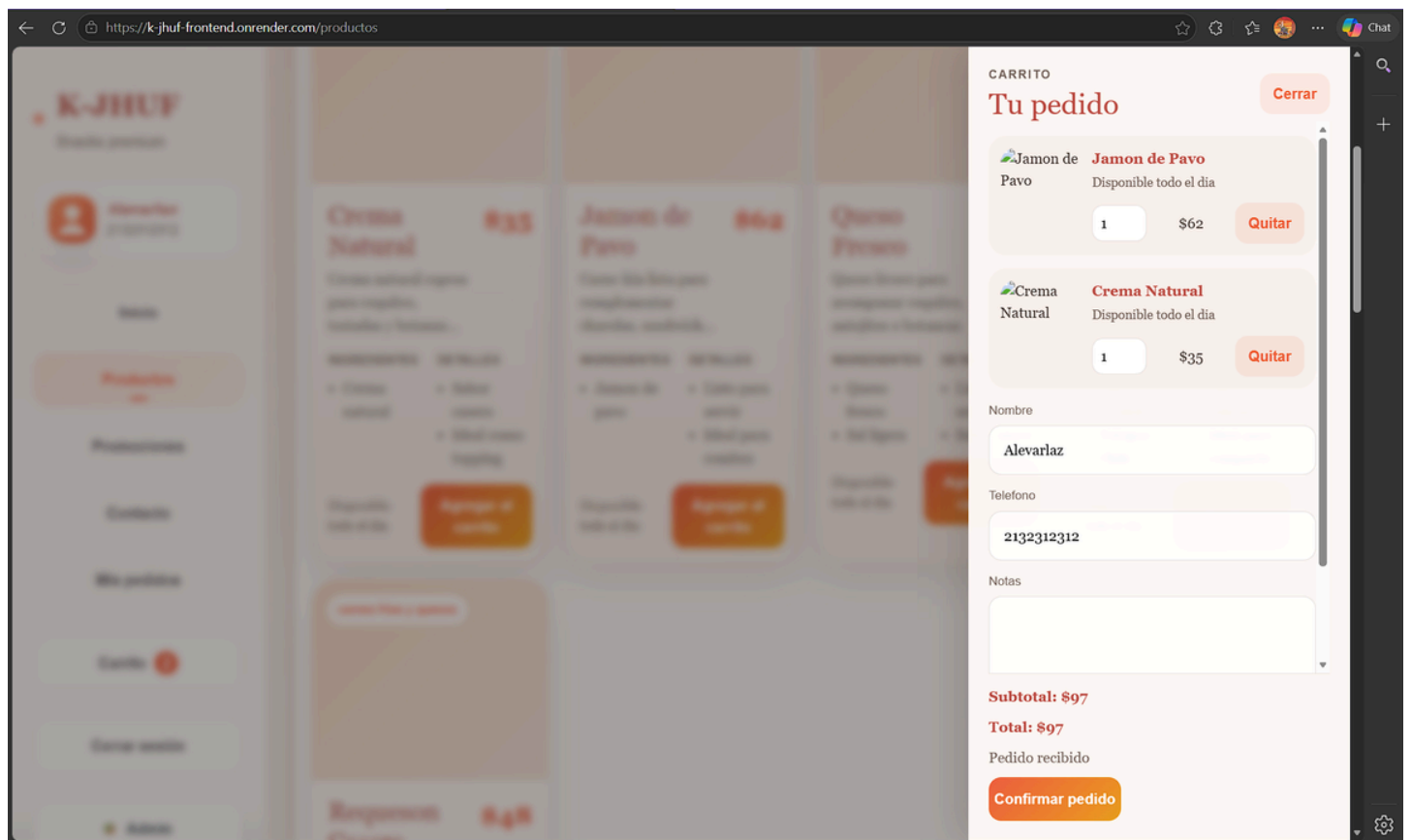
Contacto:



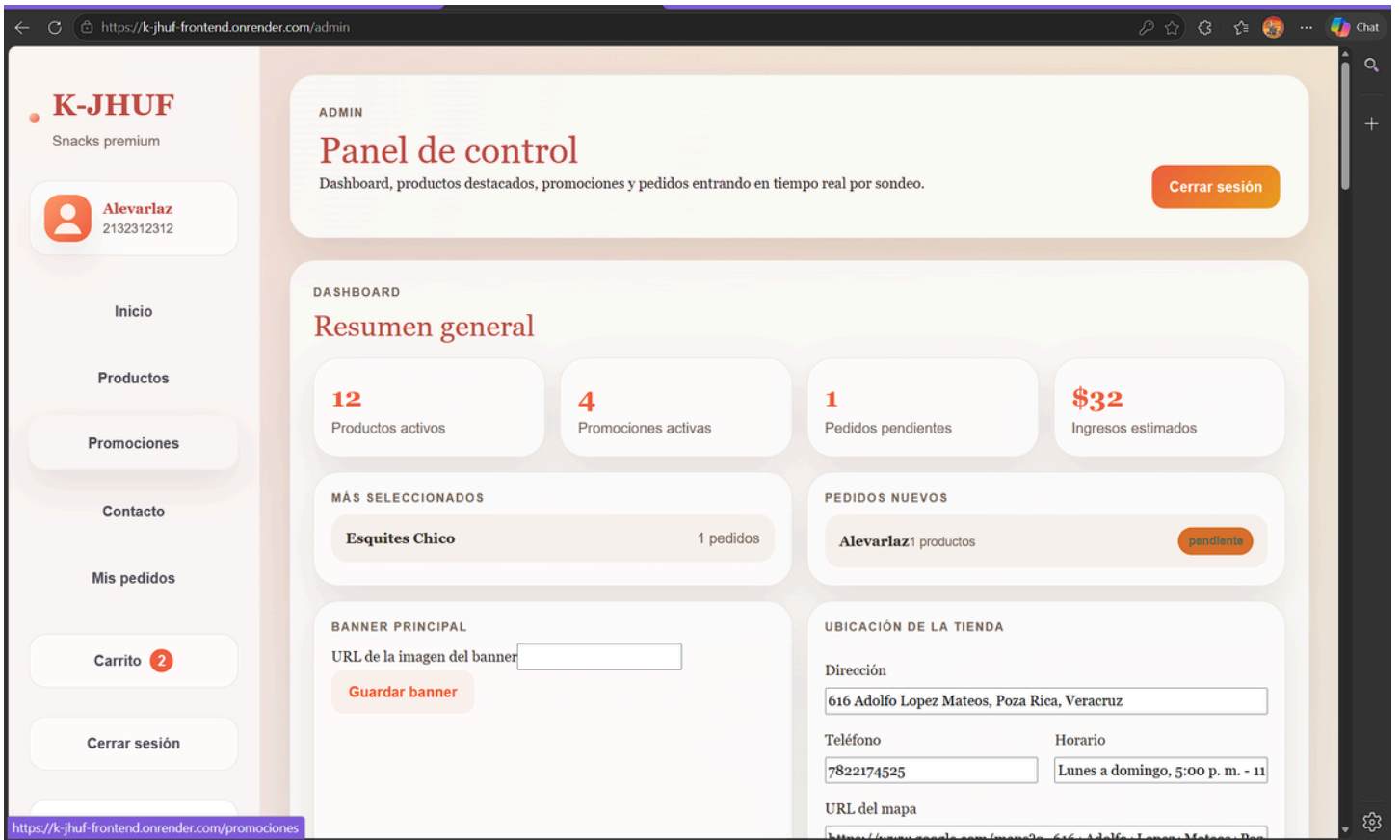
Mis pedidos:



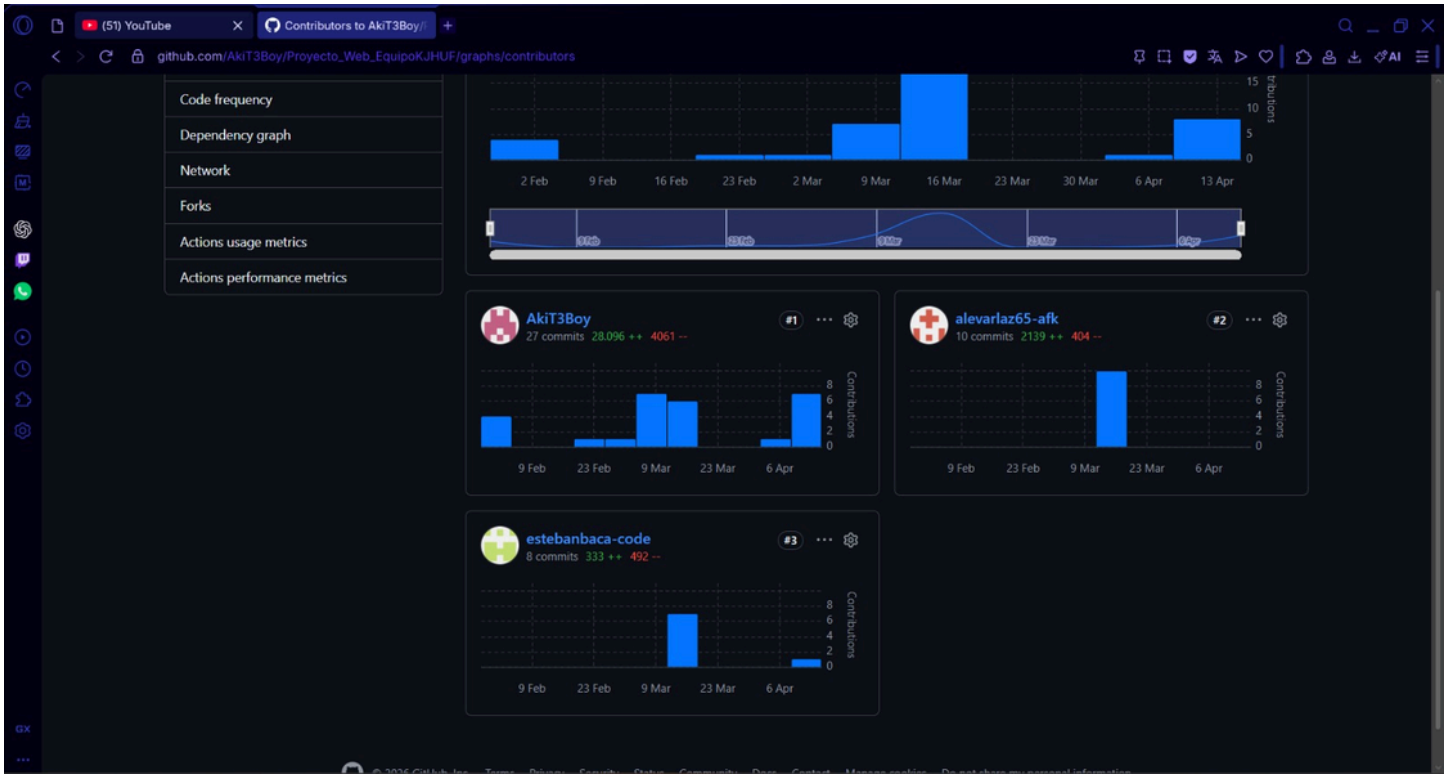
Carrito de compras:



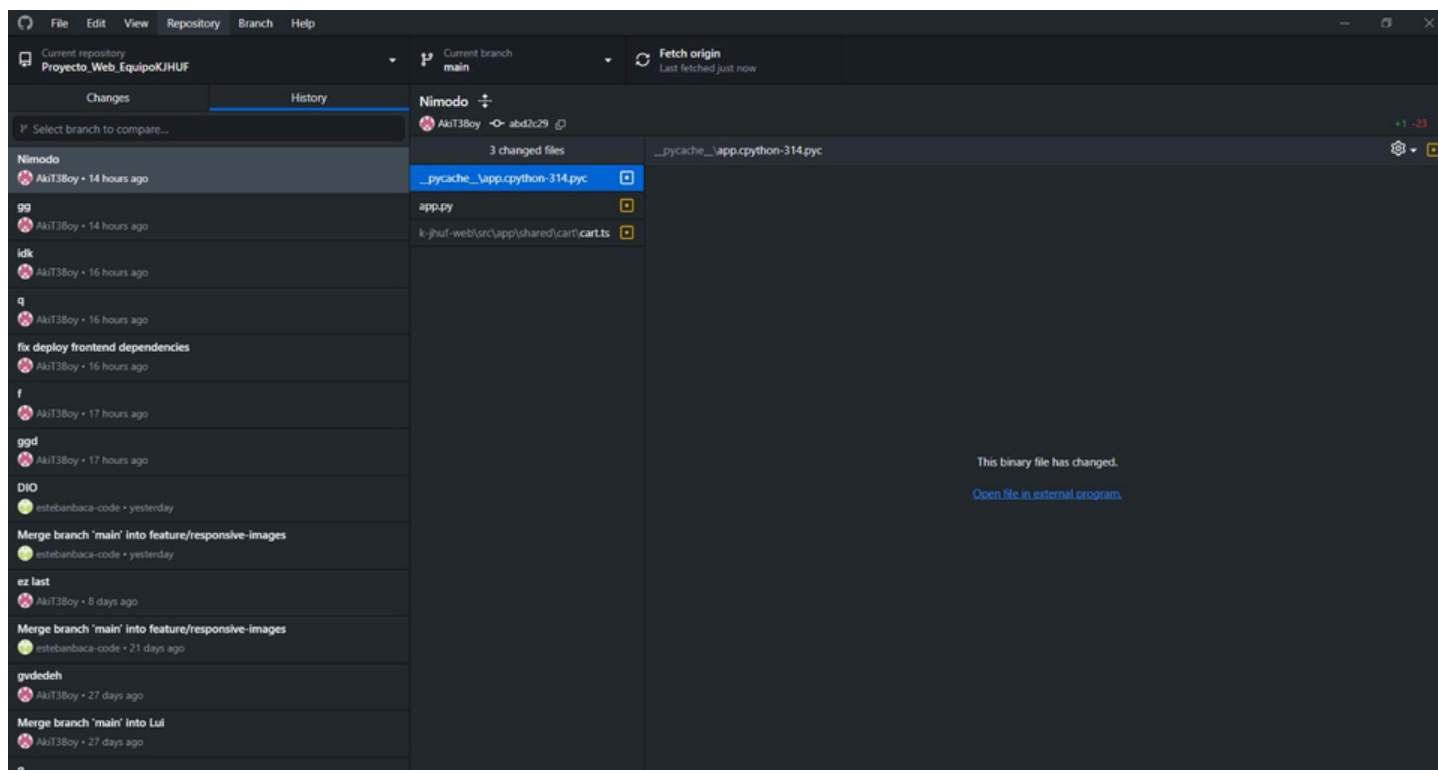
Panel de control del admin:



Commits



Ultimo commit realizado



Conclusiones

El desarrollo de la aplicación web informativa y promocional para la tienda de esquites y raspados “D LA RUIZ” permitió identificar la importancia de las tecnologías web como una herramienta fundamental para mejorar la comunicación entre los negocios locales y sus clientes. A lo largo del proyecto se analizó la situación actual del negocio, detectando la falta de presencia digital como una de las principales limitantes para la difusión de productos e información general.

Referencias

Martínez Vera, L. A. (2026). *Documentación del proyecto: Aplicación web informativa y promocional para la tienda de esquites y raspados “K-JHUF”* [Documento de trabajo]. Ingeniería en Tecnologías de la Información e Innovación Digital.

Martínez Vera, L. A. (2026). *Documentación del proyecto web K-JHUF* [Archivo PDF]. Universidad Politécnica de Pachuca.

Martínez Vera, L. A. (2026). *Descripción del modelo entidad-relación del sistema de punto de venta* [Archivo PDF]. Universidad Politécnica de Pachuca.